UNIVERSIDAD NACIONAL DE LOJA

Trabajo práctico: Desarrollo de algoritmos

Computación

Teoría de la programación

Unidad 1

Docente: Ing. José Guamán

Estudiante: David Ruiz

Curso / Paralelo: A

Fecha de entrega: 03 – 11 - 2026

Tema: Desarrollo de algoritmos: representación gráfica, pseudocódigo, prueba de escritorio y codificación en C

Objetivos

- Desarrollar los algoritmos propuestos por el docente aplicando las distintas formas de representación y construcción de soluciones: diagrama de flujo, pseudocódigo, prueba de escritorio y su implementación en el lenguaje de programación.

Instrucciones generales:

1. El trabajo se realizará de forma **individual**.
2. Se deben desarrollar **todos los algoritmos enviados** por el docente.
3. Por **cada algoritmo** se debe presentar la estructura indicada en el punto 4, respetando el orden establecido.
4. Los diagramas de flujo pueden elaborarse en herramientas como **draw.io, Flowgorithm o Lucidchart**, o a mano legible (fotografiados).
5. La codificación debe realizarse en **lenguaje C**, compilada y probada. Se debe incluir una **captura de pantalla de la salida** del programa con datos de ejemplo.
6. El documento se entregará en formato **PDF** a través de [medio de entrega].
7. La copia total o parcial de trabajos será sancionada según el reglamento institucional.
8. El docente podrá solicitar una **sustentación oral** del trabajo para verificar la autoría de las soluciones presentadas.

UNIVERSIDAD NACIONAL DE LOJA

Estructuras del trabajo

Ejercicios Avanzados

Ejercicio 296

Enunciado del algoritmo

En un hospital, diseñe un algoritmo que calcule el costo de servicio de jardinería. Lea los metros cuadrados (real), el tipo de servicio (caracter: 'C' corte, 'P' poda, 'L' limpieza, 'D' diseno), la frecuencia (caracter: 'U' unica, 'S' semanal, 'M' mensual), el número de jardineros (entero) y la complejidad (caracter: 'B' baja, 'M' media, 'A' alta). Use constantes simbólicas PRECIO_M2_CORTE = 0.5, PRECIO_M2_PODA = 1.0, PRECIO_M2_LIMPIEZA = 0.7, PRECIO_M2_DISENO = 2.0, DESC_SEMANAL = 0.10, DESC_MENSUAL = 0.20. Calcule el costo. Muestre por pantalla el detalle. Además del cálculo principal, calcule el resto de una división adicional. Agregue al final un resumen de los datos ingresados.

Análisis del problema:

Datos de entrada: Tipos de servicio: se determinan las constantes, como el precio por cada servicio dependiendo el metro cuadrado y los caracteres que se deben ingresar para escoger el tipo de servicio, las variables independientes como lo metros cuadrados a limpiar.

Proceso: Se empieza presentando un texto para que ingrese los metros cuadrados del terreno a trabajar, después para que escoja que tipo de servicio quiere, la frecuencia, el número de jardineros y la complejidad.

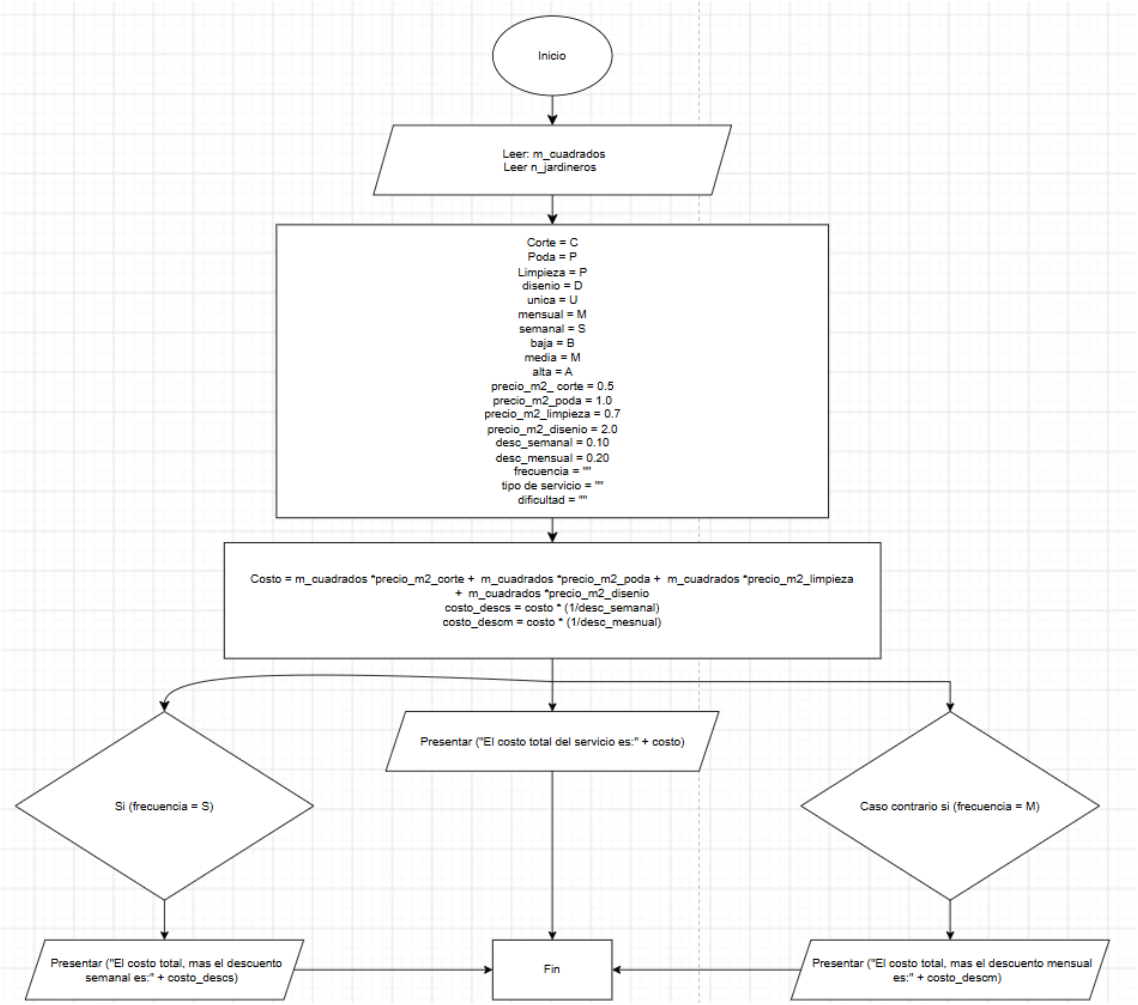
Para calcular el resultado final, se multiplica los metros cuadrados por el valor de cada designación que escogió y al final se suman para dar el valor a pagar.

Dependiendo si el cliente escoge un plan semanal o mensual, se le aplica un descuento.

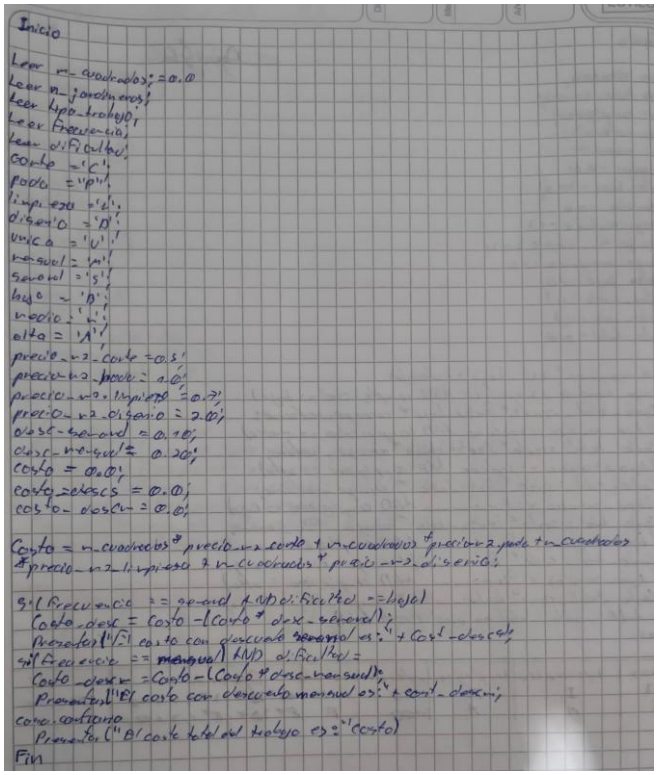
Datos de salida: Se hace conocer al cliente cual es el valor para cancelar por cada servicio

Diagrama de flujo

UNIVERSIDAD NACIONAL DE LOJA

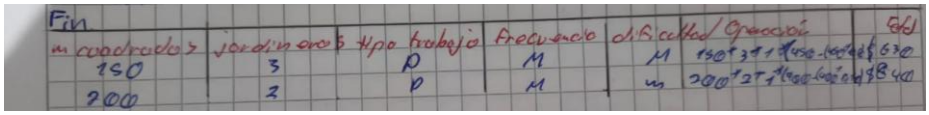


Pseudocódigo



UNIVERSIDAD NACIONAL DE LOJA

Prueba de escritorio



Codificación en lenguaje C

```c
#include <stdio.h>

int main() {
    float m_cuadrados = 0.0;
    int n_jardineros = 0;
    char tipo_trabajo = ' ';
    char frecuencia = ' ';
    char dificultad = ' ';
    char corte = 'C';
    char poda = 'P';
    char limpieza = 'L';
    char disenio = 'D';
    char unica = 'U';
    char mensual = 'M';
    char semanal = 'S';
    char baja = 'B';
    char media = 'M';
    char alta = 'A';
    float precio_m2_corte = 0.5;
    float precio_m2_poda = 1.0;
    float precio_m2_limpieza = 0.7;
    float precio_m2_disenio = 2.0;
    float desc_semanal = 0.10;
    float desc_mensual = 0.20;
    float Costo = 0.0, Costo_descs = 0.0, Costo_descm = 0.0;

    printf("Ingrese la cantidad de metros cuadrados a trabajar:\n");
    scanf(" %f", &m_cuadrados);

    printf("Ingrese el numero de jardineros:\n");
    scanf(" %d", &n_jardineros);

    printf("Ingrese el tipo de trabajo (C para corte, P para poda, L para limpieza, D para diseño):\n");
    scanf(" %s", &tipo_trabajo);

    printf("Ingrese la frecuencia del trabajo (U para unica, M para mensual, S para semanal):\n");
    scanf(" %s", &frecuencia);

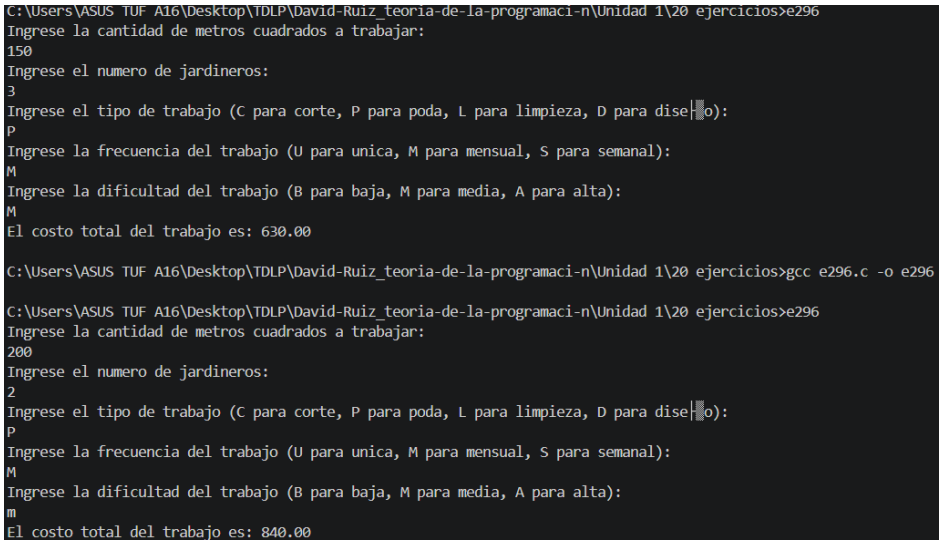
    printf("Ingrese la dificultad del trabajo (B para baja, M para media, A para alta):\n");
    scanf(" %s", &dificultad);

    Costo = m_cuadrados * precio_m2_corte + m_cuadrados * precio_m2_poda + m_cuadrados * precio_m2_limpieza + m_cuadrados * precio_m2_disenio;
```

```c
    if (frecuencia == semanal) {
        Costo_descs = Costo - (Costo * desc_semanal);
        printf("El costo con descuento semanal es: %.2f\n", Costo_descs);
    } else if (frecuencia == mensual) {
        Costo_descm = Costo - (Costo * desc_mensual);
        printf("El costo con descuento mensual es: %.2f\n", Costo_descm);
    } else {
        printf("El costo total del trabajo es: %.2f\n", Costo);
    }

}
```

Salida del programa



Conclusiones del algoritmo

El algoritmo evalúa correctamente el valor según el tipo de servicio que se escoge, mas determinar los descuentos que se aplican dependiendo la frecuencia que se pide del servicio.

Ejercicio 297

Enunciado del algoritmo

En una tienda de ropa, construya un algoritmo que controle el acceso de personal. Lea el ID del empleado (entero), la hora (entero 0-23), el día de la semana (caracter), el departamento (caracter: 'A' administracion, 'P' produccion, 'V' ventas, 'I' informatica) y el nivel de acceso (entero 1-3). Use constantes simbólicas HORA_ENTRADA = 6, HORA_SALIDA = 22, NIVEL_MIN_ADMIN = 3, NIVEL_MIN_PRODUCCION = 2. Evalúe las expresiones lógicas compuestas para autorizar el acceso. Muestre por pantalla la decisión. Además del cálculo principal, calcule el resto de una division adicional. Asegurese de que las expresiones logicas esten bien parentizadas.

Análisis del problema

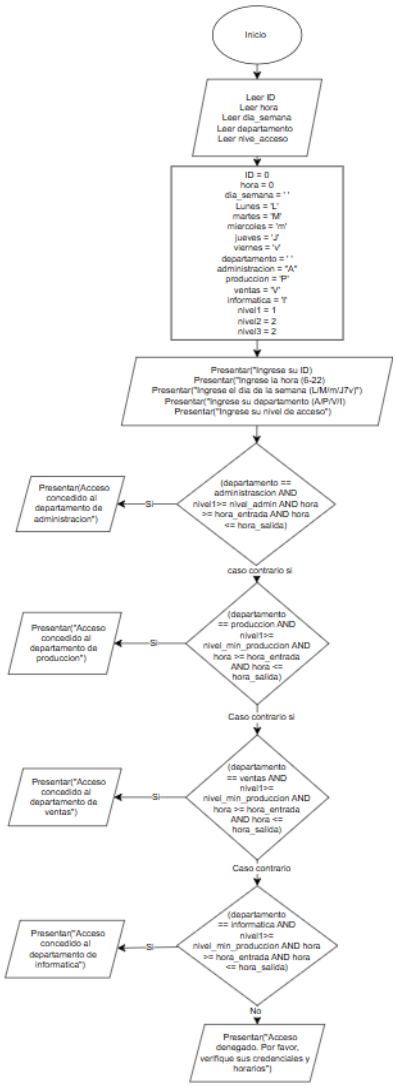
Datos de entrada: Leer el ID del empleado, mas la hora, el dia de la semana, el departemanto y nivel de acceso, las constantes simbolicas de la hora de entrada, hora de salida, el nivel mínimo para ingresar y el nivel del administrador

Proceso: Evaluar dentro los datos ingresados para poder dar acceso o denegarlo

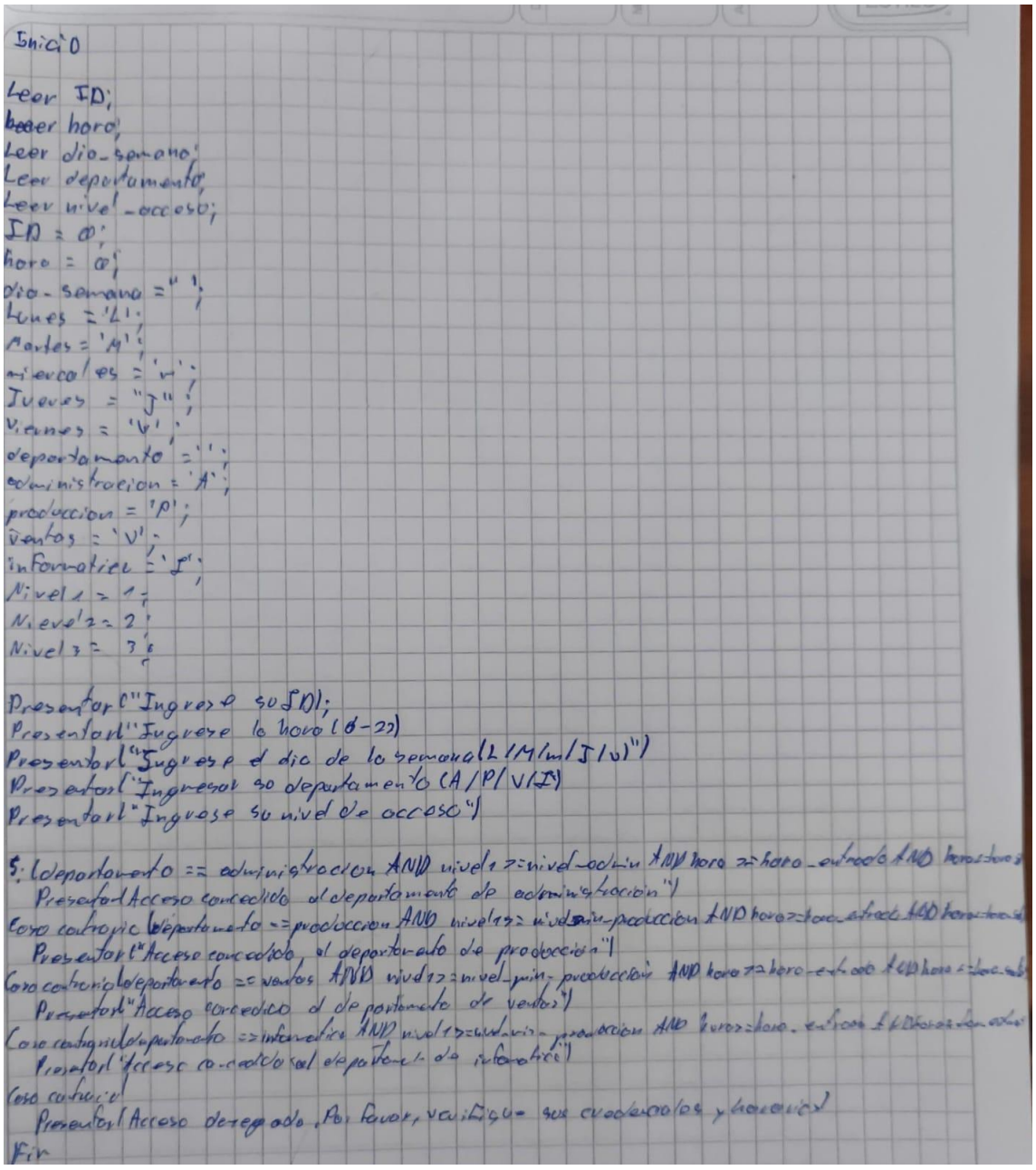
Datos de salida: Despues de haber evaluado, presentar si el acceso ha sido permitido o denegado

Diagrama de flujo

UNIVERSIDAD NACIONAL DE LOJA



Pseudocódigo



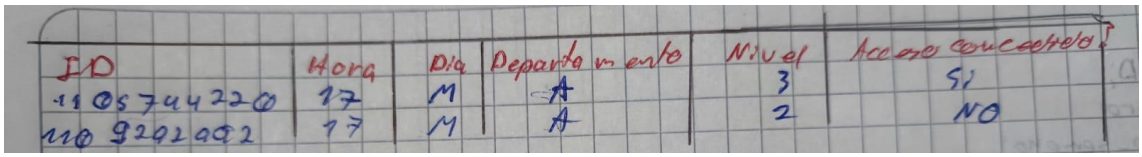
Inicio

Leer ID;
Leer hora;
Leer dia_semana;
Leer departamento;
Leer nivel_acceso;
ID = 0;
hora = 0;
dia_semana = " ";
Lunes = 'L';
Martes = 'M';
miercoles = 'm';
Jueves = "J";
Viernes = 'V';
departamento = ' ';
administracion = 'A';
produccion = 'P';
Ventas = 'V';
Informatica = 'I';
Nivel 1 = 1;
Nivel 2 = 2;
Nivel 3 = 3;

Presentar("Ingrese su ID");
Presentar("Ingrese la hora (6-22)
Presentar("Ingrese el dia de la semana(L/M/m/J/V)")
Presentar("Ingrese su departamento (A/P/V/I)
Presentar("Ingrese su nivel de acceso")

Si(departamento == administracion AND nivel 3 = nivel_acceso AND hora >= hora_entrada AND hora
Presentar("Acceso concedido al departamento de administracion")
Caso contrario(departamento == produccion AND nivel 2 = nivel_acceso AND hora >= hora_entrada AND hora
Presentar("Acceso concedido al departamento de produccion")
Caso contrario(departamento == ventas AND nivel 2 = nivel_acceso AND hora >= hora_entrada AND hora
Presentar("Acceso concedido al departamento de ventas")
Caso contrario(departamento == informatica AND nivel 3 = nivel_acceso AND hora >= hora_entrada AND hora
Presentar("Acceso concedido al departamento de informatica")
Caso contrario
Presentar("Acceso denegado. Por favor, verifique sus credenciales y horario")
Fin

Prueba de escritorio



ID	Hora	Dia	Departamento	Nivel	Acceso concedido?
1105744220	17	M	A	3	Si
1109292902	17	M	A	2	NO

Codificación en lenguaje C

```c
#include <stdio.h>

int main(){
    int ID = 0;
    int hora = 0;
    char dia_semanana = ' ';
    char administracion = 'A';
    char produccion = 'P';
    char ventas = 'V';
    char informatica = 'I';
    char Lunes = 'L';
    char martes = 'M';
    char miercoles = 'm';
    char jueves = 'J';
    char viernes = 'v';
    char departamento = ' ';
    int nivel1 = 1;
    int nivel2 = 2;
    int nivel3 = 2;
    int hora_entrada = 6;
    int hora_salida = 22;
    int nivel_admin = 3;
    int nivel_min_produccion = 2;

    printf("Ingrese su ID: ");
    scanf("%d", &ID);
    printf("Ingrese la hora (6-22): ");
    scanf("%d", &hora);
    printf("Ingrese el día de la semana (L/M/m/J/v): ");
    scanf(" %c", &dia_semanana);
    printf("Ingrese su departamento (A/P/V/I): ");
    scanf(" %c", &departamento);
    printf("Ingrese su nivel de acceso: ");
    scanf("%d", &nivel1);

    if(departamento == administracion && nivel1 >= nivel_admin && hora >= hora_entrada && hora <= hora_salida){
        printf("Acceso concedido al departamento de administración.\n");
    } else if(departamento == produccion && nivel1 >= nivel_min_produccion && hora >= hora_entrada && hora <= hora_salida){
        printf("Acceso concedido al departamento de producción.\n");
    } else if(departamento == ventas && nivel1 >= nivel_min_produccion && hora >= hora_entrada && hora <= hora_salida){
        printf("Acceso concedido al departamento de ventas.\n");
    } else if(departamento == informatica && nivel1 >= nivel_min_produccion && hora >= hora_entrada && hora <= hora_salida){
        printf("Acceso concedido al departamento de informática.\n");
    } else {
        printf("Acceso denegado. Por favor, verifique sus credenciales y horarios.\n");
    }
    return 0;
}
```

Salida del programa

```
C:\Users\ASUS TUF A16\Desktop\TDLP\David-Ruiz_teoria-de-la-programaci-n\Unidad 1\20 ejercicios>e297
Ingrese su ID: 1105744220
Ingrese la hora (6-22): 17
Ingrese el d├¡a de la semana: M
Ingrese su departamento: A
Ingrese su nivel de acceso: 3
Acceso concedido al departamento de administraci├│n.

C:\Users\ASUS TUF A16\Desktop\TDLP\David-Ruiz_teoria-de-la-programaci-n\Unidad 1\20 ejercicios>gcc e297.c -o e297

C:\Users\ASUS TUF A16\Desktop\TDLP\David-Ruiz_teoria-de-la-programaci-n\Unidad 1\20 ejercicios>e297
Ingrese su ID: 1109292992
Ingrese la hora (6-22): 17
Ingrese el d├¡a de la semana (L/M/m/J/v): M
Ingrese su departamento (A/P/V/I): A
Ingrese su nivel de acceso: 2
Acceso denegado. Por favor, verifique sus credenciales y horarios.
```

Conclusiones del algoritmo

Este algoritmo ayuda a descifrar correctamente si las credenciales del usuario son correctas y dar acceso al departamento correspondiente

Ejercicio 298

Enunciado del algoritmo

En una farmacia, cree un algoritmo que calcule el costo de un servicio de fontanería. Lea la duración (real, horas), el tipo de servicio (caracter: 'R' reparacion, 'I' instalacion, 'M' mantenimiento), la urgencia (caracter: 'N' normal, 'U' urgente, 'E'

emergencia), los materiales (real) y la complejidad (caracter: 'B' baja, 'M' media, 'A' alta). Use constantes simbólicas COSTO_HORA = 25.0, RECARGO_URGENTE = 1.5, RECARGO_EMERGENCIA = 2.5, FACTOR_REPARACION = 1.0, FACTOR_INSTALACION = 1.3, FACTOR_MANTENIMIENTO = 0.9. Calcule el costo total. Muestre por pantalla el detalle. Además del cálculo principal, calcule el resto de una división adicional. Identifique explícitamente los datos de entrada y la información de salida.

Análisis del problema

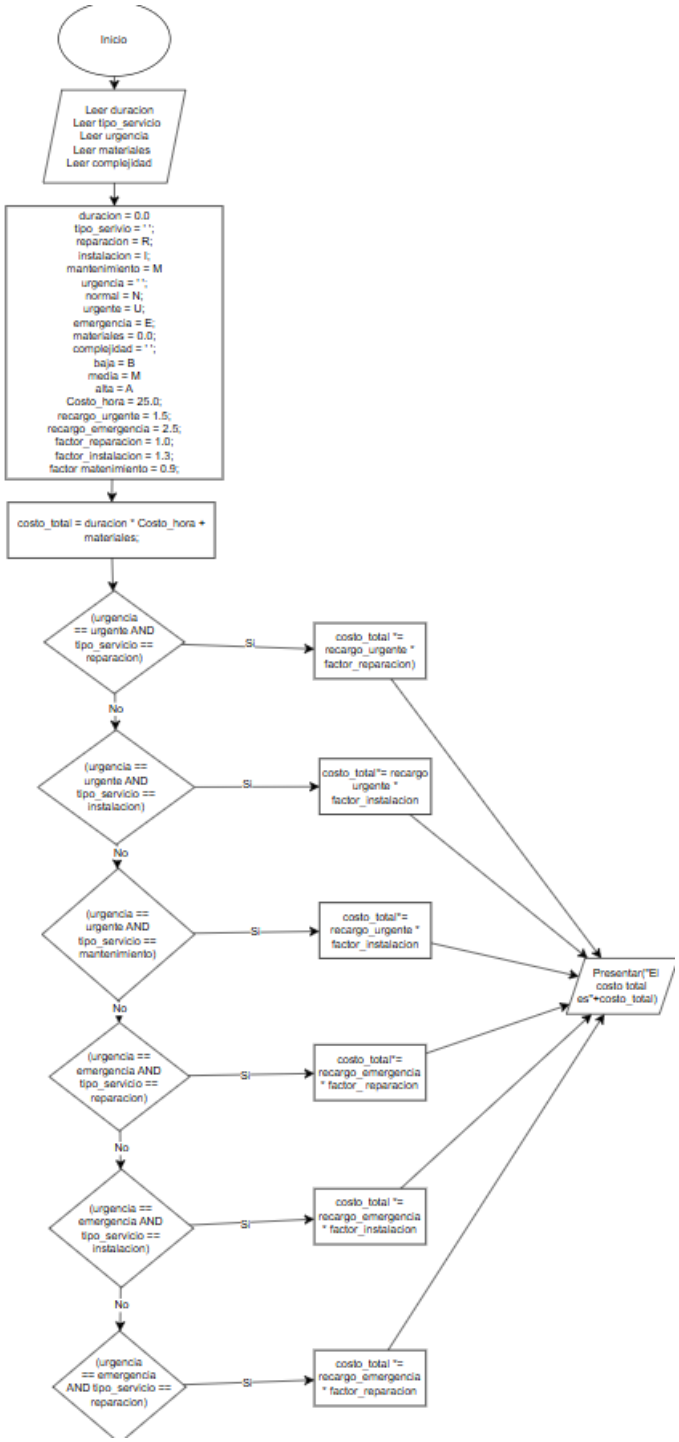
Datos de entrada:Leer el tipo de servicio, las horas de duración, mas la urgencia, el valor de los materiales y la complejidad, mas las contantes del costo por hora, recargo por urgencia y emergencia, mas factor de reparación, insalacion y mantenimiento

Proceso: Multiplicar las horas ingresadas por el costo por hora y después evaluar, dependiendo que tipo de servicio seleccione y la urgencia, se multiplicara por el factor correspondiente para al final sumar el costo de los materiales

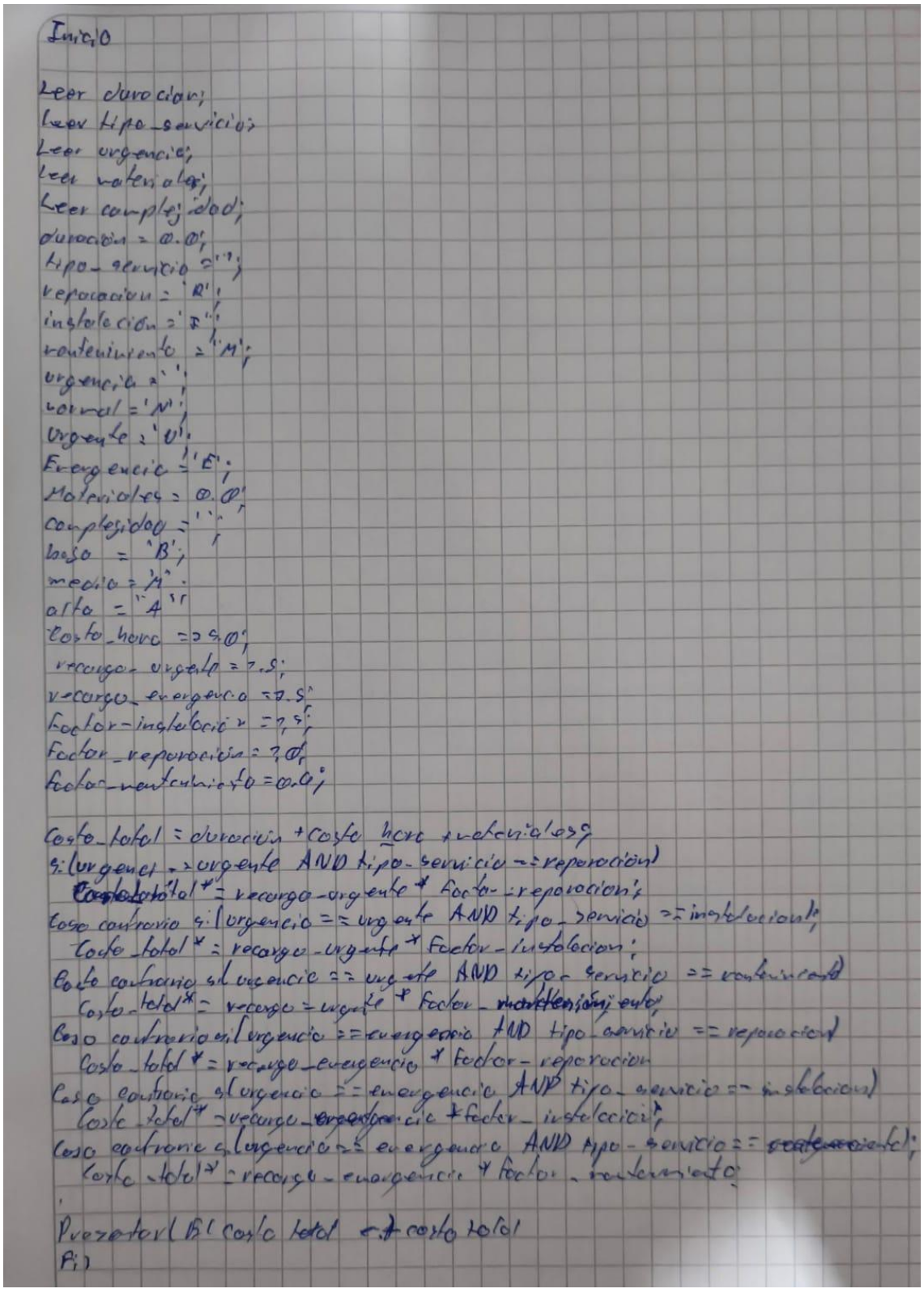
Datos de salida: Se presentará el costo total del servicio escogido

Diagrama de flujo

UNIVERSIDAD NACIONAL DE LOJA



Pseudocódigo



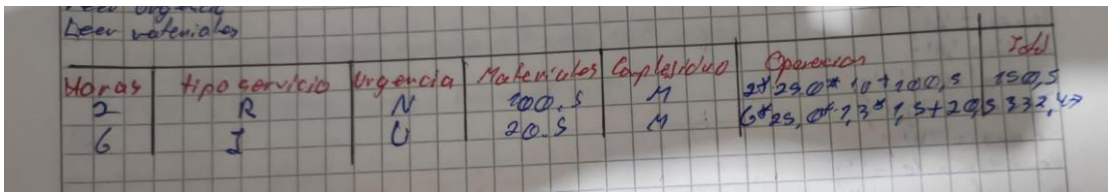
Inicio

Leer duracion;
Leer tipo_servicio;
Leer urgencia;
Leer materiales;
Leer complejidad;
duracion = 0.0;
tipo_servicio = ' ';
reparacion = 'R';
instalacion = 'I';
mantenimiento = 'M';
urgencia = ' ';
normal = 'N';
Urgente = 'U';
Emergencia = 'E';
Materiales = 0.0;
complejidad = ' ';
bajo = 'B';
medio = 'M';
alto = 'A';
costo_hora = 25.0;
recargo_urgente = 1.5;
recargo_emergencia = 2.5;
factor_instalacion = 1.5;
factor_reparacion = 1.0;
factor_mantenimiento = 0.8;

costo_total = duracion * costo_hora + materiales;
si(urgencia == urgente AND tipo_servicio == reparacion)
costo_total* = recargo_urgente * factor_reparacion;
caso contrario si(urgencia == urgente AND tipo_servicio == instalacion)
costo_total* = recargo_urgente * factor_instalacion;
caso contrario si(urgencia == urgente AND tipo_servicio == mantenimiento)
costo_total* = recargo_urgente * factor_mantenimiento;
caso contrario si(urgencia == emergencia AND tipo_servicio == reparacion)
costo_total* = recargo_emergencia * factor_reparacion
caso contrario si(urgencia == emergencia AND tipo_servicio == instalacion)
costo_total* = recargo_emergencia * factor_instalacion;
caso contrario si(urgencia == emergencia AND tipo_servicio == mantenimiento);
costo_total* = recargo_emergencia * factor_mantenimiento;

Prezetar('costo total =', costo_total)
Fin

Prueba de escritorio



Leer materiales

Horas	tipo servicio	Urgencia	Materiales	Complejidad	Operacion	Total
2	R	N	100.5	M	2*25.0*1.0+100.5	150.5
6	I	U	20.5	M	6*25.0*1.3*1.5+20.5	332.4

Codificación en lenguaje C

```c
#include <stdio.h>

int main(){
    float duracion = 0.0;
    char tipo_servicio = ' ';
    char reparacion = 'R';
    char instalacion = 'I';
    char mantenimiento = 'M';
    char urgencia = ' ';
    char normal = 'N';
    char urgente = 'U';
    char emergencia = 'E';
    float materiales = 0.0;
    char complejidad = ' ';
    char baja = 'B';
    char media = 'M';
    char alta = 'A';
    float Costo_hora = 25.0;
    float recargo_urgente = 1.5;
    float recargo_emergencia = 2.5;
    float factor_reparacion = 1.0;
    float factor_instalacion = 1.3;
    float factor_mantenimiento = 0.9;
    float costo_total = 0.0;

    printf("Ingrese la duracion del servicio en horas: ");
    scanf("%f", &duracion);
    printf("Ingrese el tipo de servicio (R/I/M): ");
    scanf(" %c", &tipo_servicio);
    printf("Ingrese la urgencia del servicio (N/U/E): ");
    scanf(" %c", &urgencia);
    printf("Ingrese el costo de los materiales: ");
    scanf("%f", &materiales);
    printf("Ingrese la complejidad del servicio (B/M/A): ");
    scanf(" %c", &complejidad);

    costo_total = duracion * Costo_hora + materiales;

    if(urgencia == urgente && tipo_servicio == reparacion){
        costo_total *= recargo_urgente * factor_reparacion;
    } else if(urgencia == urgente && tipo_servicio == instalacion){
        costo_total *= recargo_urgente * factor_instalacion;
    } else if(urgencia == urgente && tipo_servicio == mantenimiento){
        costo_total *= recargo_urgente * factor_mantenimiento;
    } else if(urgencia == emergencia && tipo_servicio == reparacion){
        costo_total *= recargo_emergencia * factor_reparacion;
    } else if(urgencia == emergencia && tipo_servicio == instalacion){
        costo_total *= recargo_emergencia * factor_instalacion;
    } else if(urgencia == emergencia && tipo_servicio == mantenimiento){
        costo_total *= recargo_emergencia * factor_mantenimiento;
    }

    printf("El costo total del servicio es: %.2f\n", costo_total);
    return 0;
}
```

Salida del programa

```
C:\Users\ASUS TUF A16\Desktop\TDLP\David-Ruiz_teoria-de-la-programaci-n\Unidad 1\20 ejercicios>e298
Ingrese la duracion del servicio en horas: 2
Ingrese el tipo de servicio (R/I/M): R
Ingrese la urgencia del servicio (N/U/E): N
Ingrese el costo de los materiales: 100.5
Ingrese la complejidad del servicio (B/M/A): M
El costo total del servicio es: 150.50

C:\Users\ASUS TUF A16\Desktop\TDLP\David-Ruiz_teoria-de-la-programaci-n\Unidad 1\20 ejercicios>e298
Ingrese la duracion del servicio en horas: 6
Ingrese el tipo de servicio (R/I/M): I
Ingrese la urgencia del servicio (N/U/E): U
Ingrese el costo de los materiales: 20.5
Ingrese la complejidad del servicio (B/M/A): M
El costo total del servicio es: 332.47
```

Conclusiones del algoritmo evalúa correctamente que factor de tipo de servicio y urgencia debe usar para multiplicar al costo total del servicio

Ejercicio 299

Enunciado del algoritmo

En un taller mecánico, diseñe un algoritmo que gestione citas médicas. Lea la especialidad (caracter: 'G' general, 'P' pediatria, 'C' cardiologia, 'N' neurologia), el tipo de paciente (caracter: 'N' nuevo, 'R' recurrente, 'U' urgente), la edad (entero), la duración estimada (real, minutos) y el dia de la semana (caracter). Use constantes simbólicas DURACION_NUEVO = 60, DURACION_RECURRENTE = 30, DURACION_URGENTE = 15, ESPERA_MAX = 7. Evalúe las expresiones lógicas para asignar prioridad. Calcule el costo estimado. Muestre por pantalla. Además del cálculo principal, calcule el resto de una división adicional. Asegúrese de inicializar todas las variables antes de usarlas.

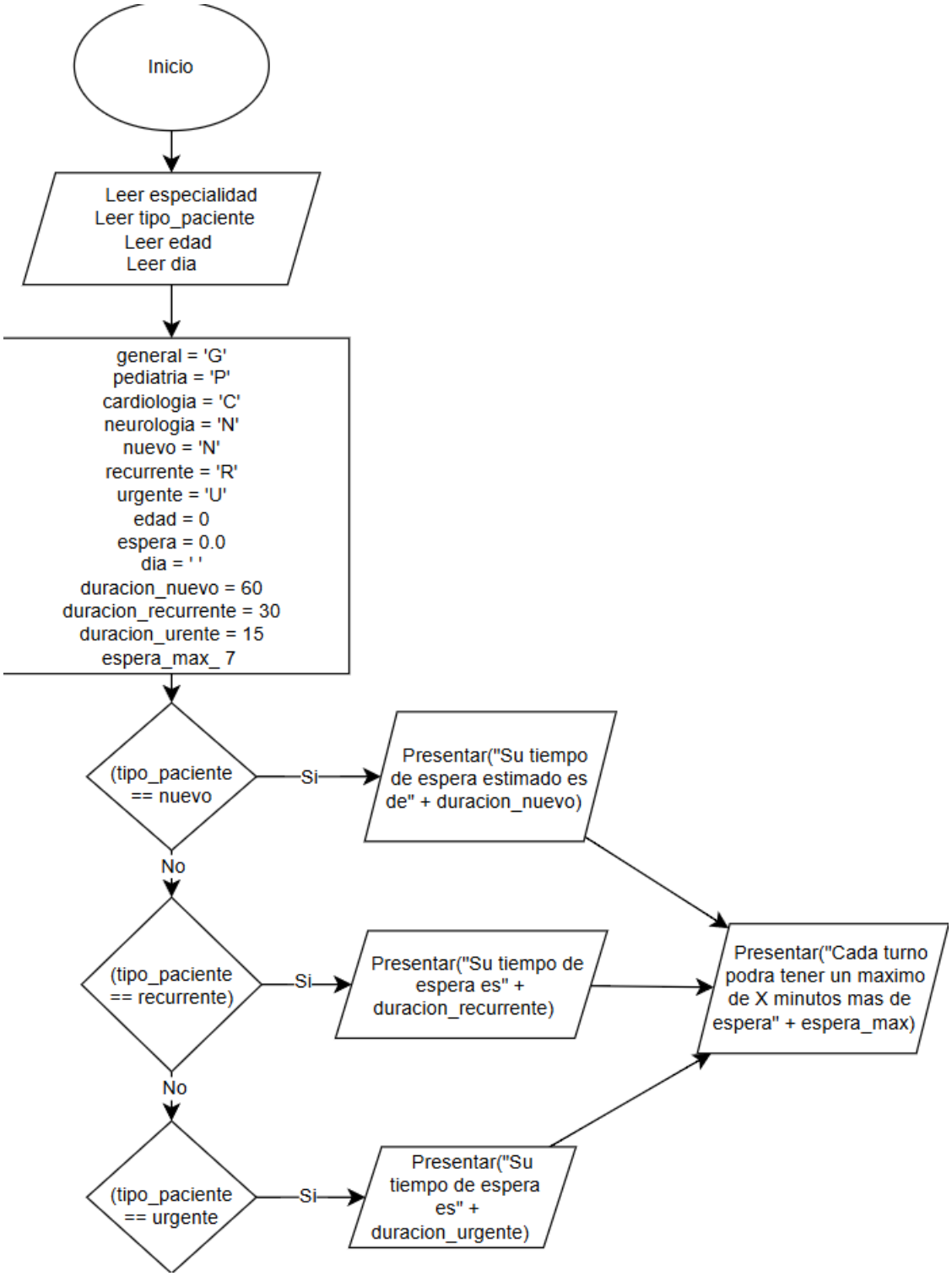
Análisis del problema

Datos de entrada: Leer la especialidad que desea, tipo de paciente, el tiempo de espera en minutos y el día de la semana

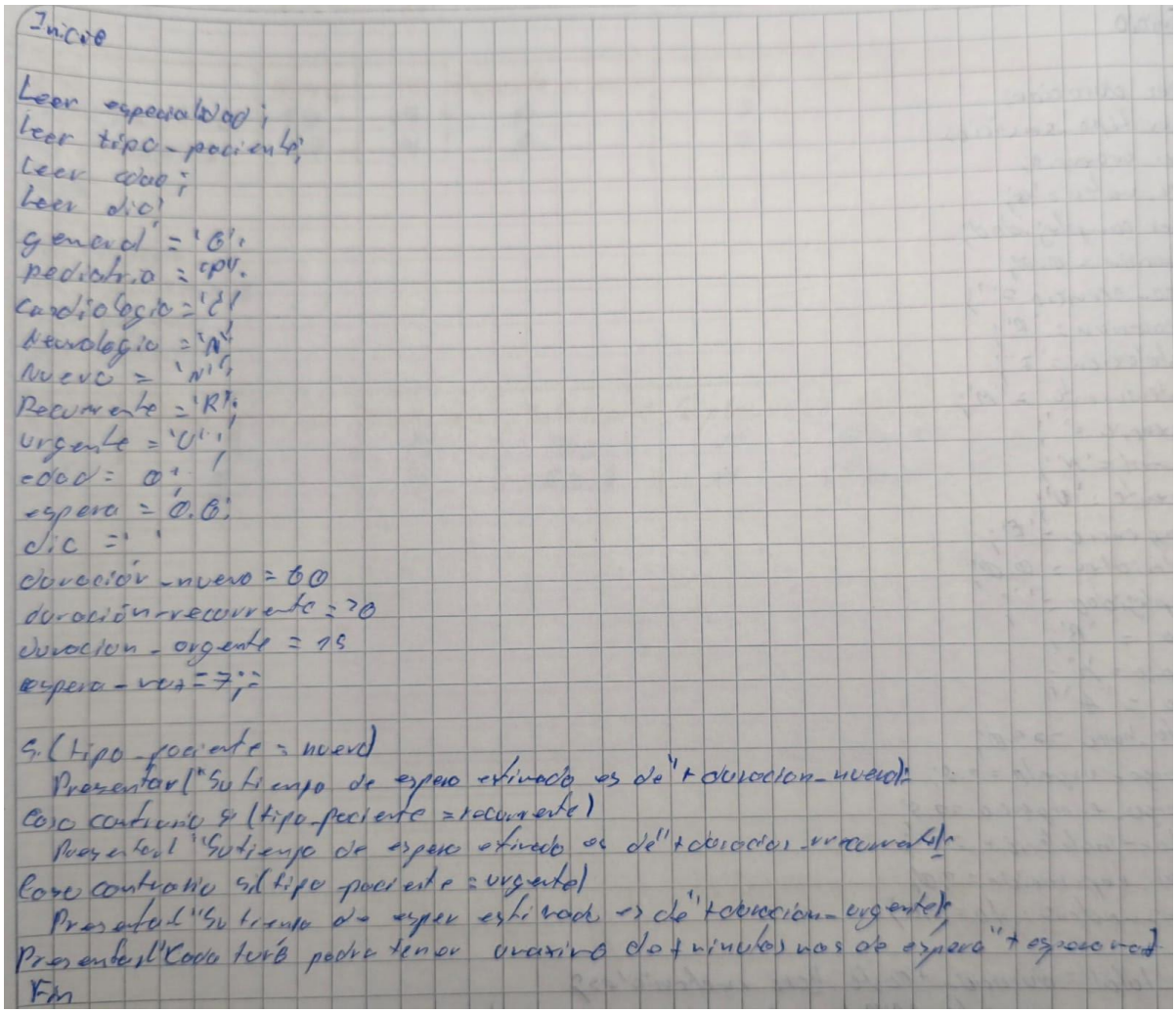
Proceso: evaluar que tipo de paciente es y determinar cuanto tiempo de espera le toca

Datos de salida: Cuanto tiempo de espera le fue asignado y el tiempo máximo de espera

Diagrama de flujo



Pseudocódigo



```
Inicio
Leer especialidad;
leer tipo_paciente;
Leer edad;
Leer dia;
general = 'G';
pediatria = 'P';
cardiologia = 'C';
Neurologia = 'N';
Nuevo = 'N';
Recurrente = 'R';
urgente = 'U';
edad = 0;
espera = 0.0;
dia = '
duracion_nuevo = 60
duracion_recurrente = 30
duracion_urgente = 15
espera_vez = 7;

Si (tipo_paciente = nuevo)
  Presentar("Su tiempo de espera estimado es de" + duracion_nuevo);
Caso contrario si (tipo_paciente = recurrente)
  Presentar("Su tiempo de espera estimado es de" + duracion_recurrente)
Caso contrario si (tipo_paciente = urgente)
  Presentar("Su tiempo de espera estimado es de" + duracion_urgente)
Presentar("Cada turno podra tener variacion de minutos) mas de espera" + espera_vez)
Fin
```

Prueba de escritorio

Especialidad	tipo de paciente	Edad	dia	duracion	duracion extra
N	R	18	L	30	+7m
P	U	20	M	15	

Codificación en lenguaje C

```c
#include <stdio.h>

int main(){
    char especialidad = ' ';
    char tipo_paciente = ' ';
    int edad = 0;
    char dia = ' ';
    char general = 'G';
    char pediatria = 'P';
    char cardiologia = 'C';
    char neurologia = 'N';
    char nuevo = 'N';
    char recurrente = 'R';
    char urgente = 'U';
    int duracion_nuevo = 60;
    int duracion_recurrente = 30;
    int duracion_urgente = 15;
    int espera_maxima = 7;

    printf("Ingrese la especialidad que necesita (G/P/C/N): ");
    scanf(" %c", &especialidad);
    printf("Ingrese el tipo de paciente que es considerado(N/R/U): ");
    scanf(" %c", &tipo_paciente);
    printf("Ingrese su edad: ");
    scanf("%d", &edad);
    printf("Ingrese el dia de la semana para la cita (L/M/m/J/v): ");
    scanf(" %c", &dia);

    if(tipo_paciente == nuevo){
        printf("Su tiempo estimado de espera es de %d minutos.\n", duracion_nuevo);
    } else if(tipo_paciente == recurrente){
        printf("Su tiempo estimado de espera es de %d minutos.\n", duracion_recurrente);
    } else if(tipo_paciente == urgente){
        printf("Su tiempo estimado de espera es de %d minutos.\n", duracion_urgente);
    }

    printf("Cada turno podra tener un maximo de %d minutos adicionales de espera.\n", espera_maxima);
    return 0;
}
```

Salida del programa

```
C:\Users\ASUS TUF A16\Desktop\TDLP\David-Ruiz_teoria-de-la-programaci-n\Unidad 1\20 ejercicios>e299
Ingrese la especialidad que necesita (G/P/C/N): N
Ingrese el tipo de paciente que es considerado(N/R/U): R
Ingrese su edad: 18
Ingrese el dia de la semana para la cita (L/M/m/J/v): L
Su tiempo estimado de espera es de 30 minutos.
Cada turno podra tener un maximo de 7 minutos adicionales de espera.

C:\Users\ASUS TUF A16\Desktop\TDLP\David-Ruiz_teoria-de-la-programaci-n\Unidad 1\20 ejercicios>e299
Ingrese la especialidad que necesita (G/P/C/N): P
Ingrese el tipo de paciente que es considerado(N/R/U): U
Ingrese su edad: 20
Ingrese el dia de la semana para la cita (L/M/m/J/v): M
Su tiempo estimado de espera es de 15 minutos.
Cada turno podra tener un maximo de 7 minutos adicionales de espera.
```

Conclusiones del algoritmo

El algoritmo determina correctamente el tiempo de espera dependiendo que tipo de paciente es considerado, dando el mensaje que puede tardar 7 minutos adicionales de lo indicado

Ejercicio 300

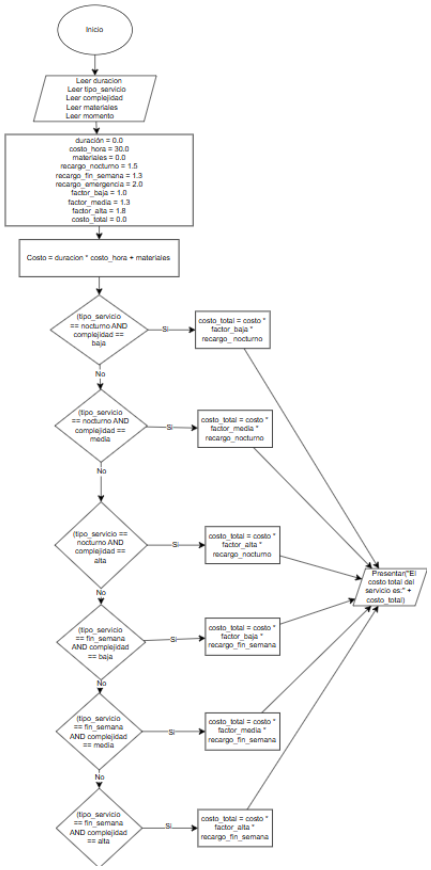
Enunciado del algoritmo

UNIVERSIDAD NACIONAL DE LOJA

En una escuela, construya un algoritmo que calcule el costo de un servicio de electricista. Lea la duración (real, horas), el tipo de servicio (carácter: 'R' reparación, 'I' instalación, 'M' mantenimiento, 'E' emergencia), la complejidad (caracter: 'B' baja, 'M' media, 'A' alta), los materiales (real) y el momento (caracter: 'D' diurno, 'N' nocturno, 'F' fin de semana). Use constantes simbólicas COSTO_HORA = 30.0, RECARGO_NOCTURNO = 1.5, RECARGO_FIN_SEMANA = 1.3, RECARGO_EMERGENCIA = 2.0, FACTOR_BAJA = 1.0, FACTOR_MEDIA = 1.3, FACTOR_ALTA = 1.8. Calcule el costo. Muestre por pantalla el detalle. Además del cálculo principal, calcule el resto de una división adicional. Incluya una variable auxiliar para llevar un contador de operaciones.

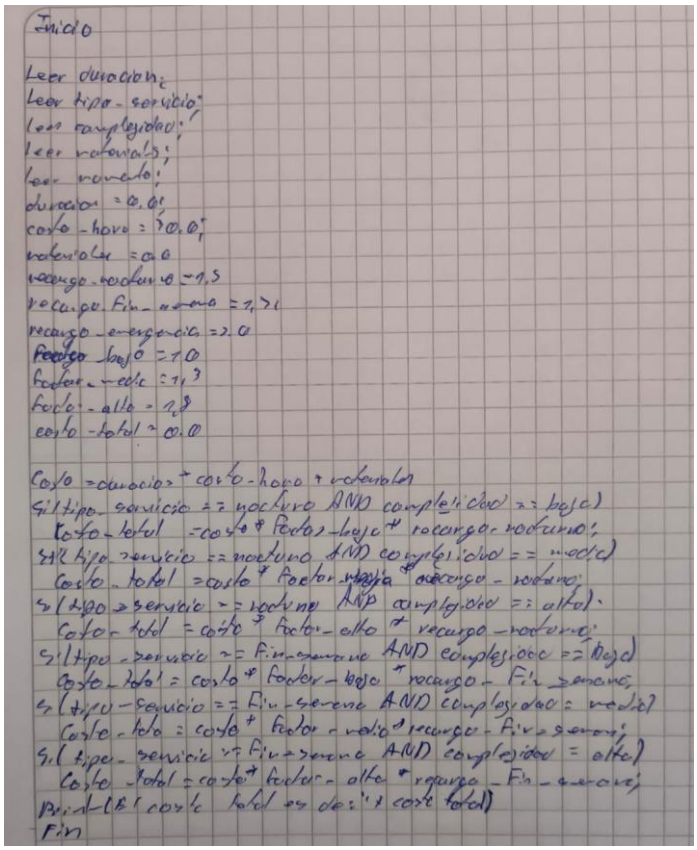
Análisis del algoritmo

Diagrama de flujo

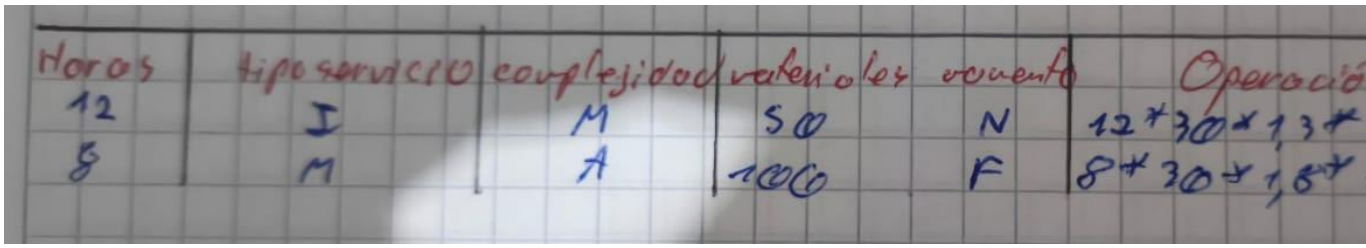


Pseudocódigo

UNIVERSIDAD NACIONAL DE LOJA



Pruebas de escritorio



Codificación en lenguaje C

```c
#include <stdio.h>

int main(){
    float horas = 0.0;
    char tipo_servicio = ' ';
    char reparacion = 'R';
    char instalacion = 'I';
    char mantenimiento = 'M';
    char emergencia = 'E';
    char complejidad = ' ';
    char bajo = 'B';
    char medio = 'M';
    char alto = 'A';
    float materiales = 0.0;
    char momento = ' ';
    char diurno = 'D';
    char nocturno = 'N';
    char fin_semana = 'F';
    float costo_hora = 30.0;
    float recargo_nocturno = 1.5;
    float recargo_fin_semana = 1.3;
    float recargo_emergencia = 2.0;
    float recargo_bajo = 1.0;
    float recargo_medio = 1.3;
    float recargo_alto = 1.8;
    float costo_total = 0.0;

    printf("Ingrese la cantidad de horas del servicio: ");
    scanf("%f", &horas);
    printf("Ingrese el tipo de servicio (R/I/M/E): ");
    scanf(" %c", &tipo_servicio);
    printf("Ingrese la complejidad del servicio (B/M/A): ");
    scanf(" %c", &complejidad);
    printf("Ingrese el momento del servicio (D/N/F): ");
    scanf(" %c", &momento);
    printf("Ingrese el costo de los materiales: ");
    scanf("%f", &materiales);

    costo_total = horas * costo_hora + materiales;

    if(momento == nocturno && complejidad == bajo){
        costo_total *= recargo_nocturno * recargo_bajo;
    } else if(momento == nocturno && complejidad == medio){
        costo_total *= recargo_nocturno * recargo_medio;
    } else if(momento == nocturno && complejidad == alto){
        costo_total *= recargo_nocturno * recargo_alto;
    } else if(momento == fin_semana && complejidad == bajo){
        costo_total *= recargo_fin_semana * recargo_bajo;
    } else if(momento == fin_semana && complejidad == medio){
        costo_total *= recargo_fin_semana * recargo_medio;
    } else if(momento == fin_semana && complejidad == alto){
        costo_total *= recargo_fin_semana * recargo_alto;
    } else if(momento == emergencia && complejidad == bajo){
        costo_total *= recargo_emergencia * recargo_bajo;
    } else if(momento == emergencia && complejidad == medio){
        costo_total *= recargo_emergencia * recargo_medio;
    } else if(momento == emergencia && complejidad == alto){
        costo_total *= recargo_emergencia * recargo_alto;
    }
    printf("El costo total del servicio es: $%.2f\n", costo_total);
    return 0;
}
```

Salida del programa

```
C:\Users\ASUS TUF A16\Desktop\TDLP\David-Ruiz_teoria-de-la-programaci-n\Unidad 1\20 ejercicios>e300
Ingrese la cantidad de horas del servicio: 12
Ingrese el tipo de servicio (R/I/M/E): I
Ingrese la complejidad del servicio (B/M/A): M
Ingrese el momento del servicio (D/N/F): N
Ingrese el costo de los materiales: 50
El costo total del servicio es: $799.50

C:\Users\ASUS TUF A16\Desktop\TDLP\David-Ruiz_teoria-de-la-programaci-n\Unidad 1\20 ejercicios>e300
Ingrese la cantidad de horas del servicio: 8
Ingrese el tipo de servicio (R/I/M/E): M
Ingrese la complejidad del servicio (B/M/A): A
Ingrese el momento del servicio (D/N/F): F
Ingrese el costo de los materiales: 100
El costo total del servicio es: $795.60
```

Conclusión del algoritmo

Ejercicios Medios

Ejercicio 46

Enunciado del algoritmo

En un taller mecánico, cree un algoritmo que lea una base (real) y un exponente (entero positivo). Calcule la potencia con la expresión aritmética (base ^ exponente) y asígnela a resultado. Evalúe la expresión lógica (exponente > 0 AND base != 0). Muestre por pantalla el resultado de la potencia. Además del cálculo principal, determine si el resultado es positivo o negativo. Agregue un mensaje indicando el tipo de operación realizada.

Análisis del problema

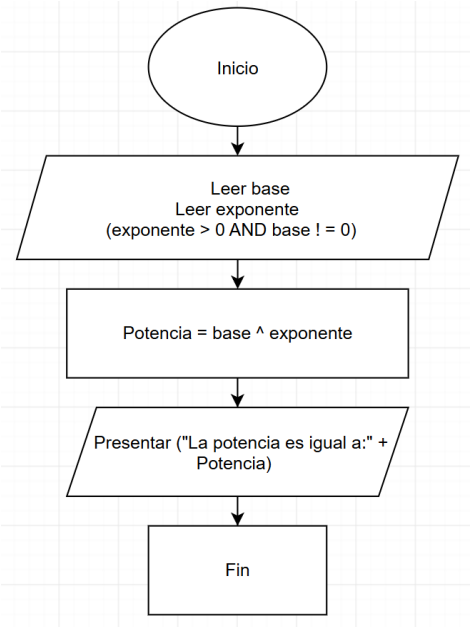
Datos de entrada: Un numero base que sea diferente de cero y un exponente que sea mayor a cero.

Proceso: Se permite al usuario ingresar un numero base y un exponente al que esté elevado para que realice la multiplicación.

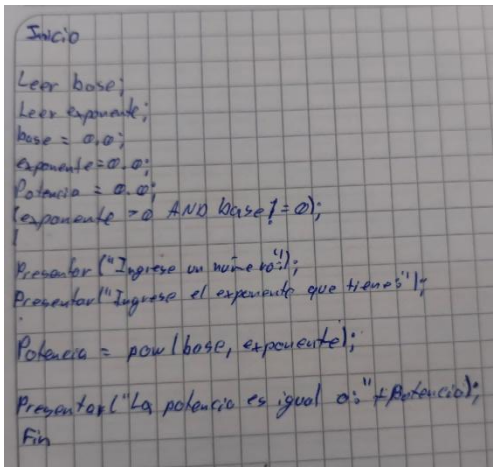
Datos de salida: La multiplicación resultante del número base al exponente que estaba elevado

Diagrama de flujo

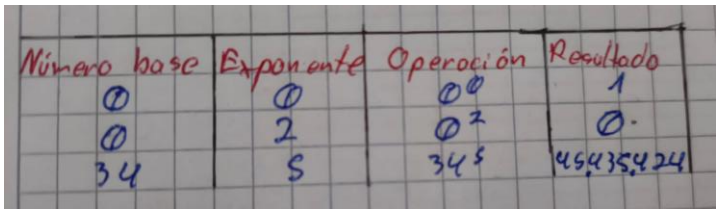
UNIVERSIDAD NACIONAL DE LOJA



Pseudocódigo



Prueba de escritorio



Codificación en lenguaje C

```c
#include <stdio.h>
#include <math.h>

int main(){
    float base = 0.0;
    float exponente = 0.0;
    float Potencia = 0.0;
    (exponente > 0 && base != 0);

    printf("Ingrese un numero:");
    scanf("%f", &base);

    printf("Ingrese el exponente que tiene:");
    scanf("%f", &exponente);

    Potencia = pow(base, exponente);

    printf("La potencia es igual a: %f", Potencia);

    return 0;
}
```

Salida del programa

```
C:\Users\ASUS TUF A16\Desktop\TDLP\David-Ruiz_teoria-de-la-programaci-n\Unidad 1\20 ejercicios>e46
Ingrese un numero:0
Ingrese el exponente que tiene:0
La potencia es igual a: 1.000000
C:\Users\ASUS TUF A16\Desktop\TDLP\David-Ruiz_teoria-de-la-programaci-n\Unidad 1\20 ejercicios>e46
Ingrese un numero:0
Ingrese el exponente que tiene:2
La potencia es igual a: 0.000000
C:\Users\ASUS TUF A16\Desktop\TDLP\David-Ruiz_teoria-de-la-programaci-n\Unidad 1\20 ejercicios>e46
Ingrese un numero:34
Ingrese el exponente que tiene:5
La potencia es igual a: 45435424.000000
C:\Users\ASUS TUF A16\Desktop\TDLP\David-Ruiz_teoria-de-la-programaci-n\Unidad 1\20 ejercicios>
```

Conclusiones del algoritmo

Este algoritmo se plantea como base para resolver exponentes y mas adelante operaciones y ecuación exponenciales.

Ejercicio 47

Enunciado del algoritmo

En una librería, construya un algoritmo que lea una distancia en millas (real). Use una constante simbólica MILLA_KM = 1.60934. Calcule la equivalencia en kilómetros con la expresión aritmética (millas * MILLA_KM) y asígnela a km. Evalúe la expresión relacional (km > 0). Muestre por pantalla 'X millas equivalen a Y km'. Además del cálculo principal, calcule el logaritmo en base 10 del resultado. Muestre al final los valores de todas las variables auxiliares.

Análisis del problema

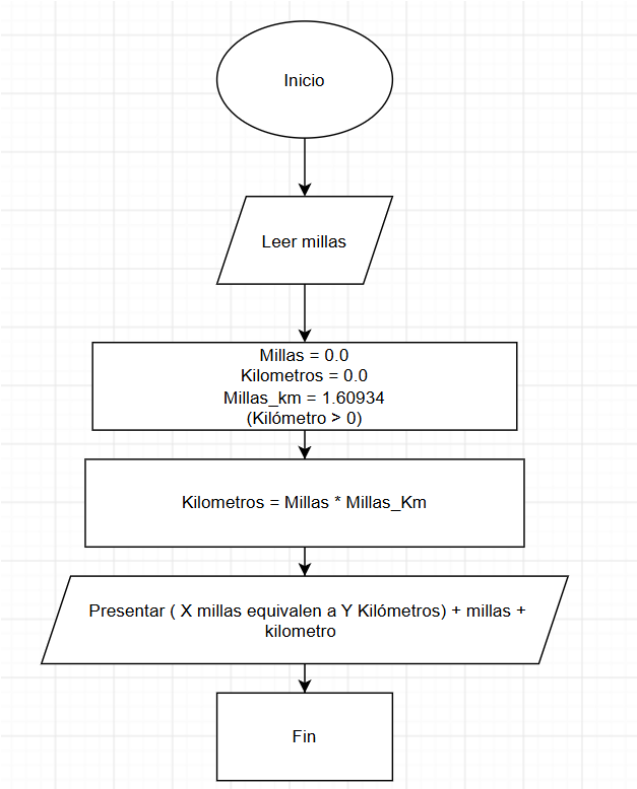
Datos de entrada: Millas a medir, cuanto equivale una milla en kilómetros, que los kilómetros sean mayores a cero.

UNIVERSIDAD NACIONAL DE LOJA

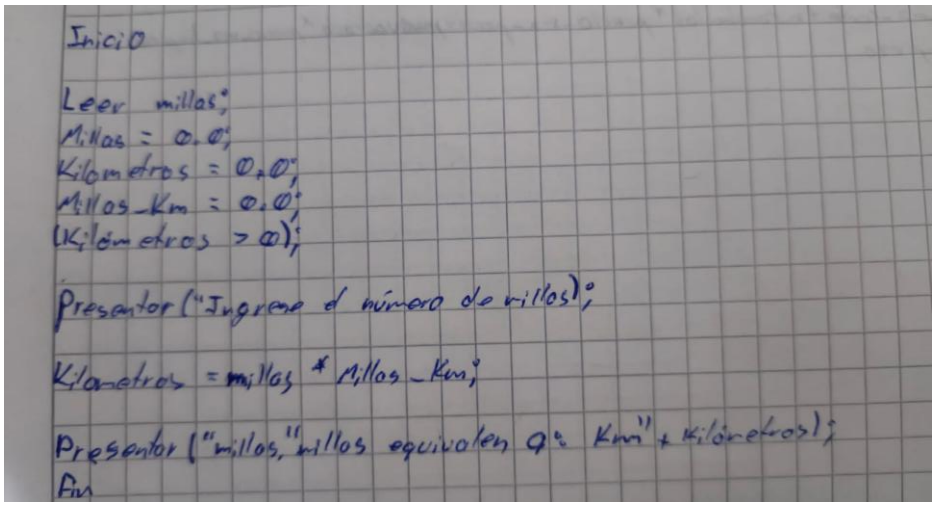
Proceso: Multiplicar las millas ingresadas por el valor equivalente a un kilómetro y transformar

Datos de salida: Cuanto equivale una cantidad de millas en kilómetros

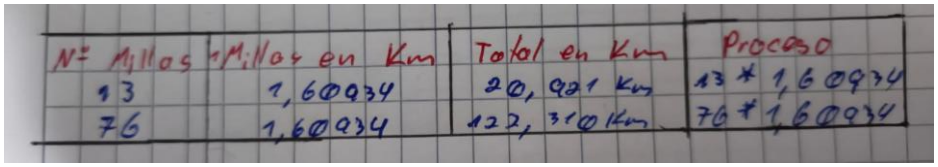
Diagrama de flujo



Pseudocódigo



Prueba de escritorio



N° Millas	Millas en Km	Total en Km	Proceso
13	1,60934	20,921 Km	13 * 1,60934
76	1,60934	122,310 Km	76 * 1,60934

Codificación en lenguaje C

```c
#include <stdio.h>

int main(){
    float millas = 0.0;
    float kilometros = 0.0;
    float Millas_Km = 1.60934;

    (kilometros > 0);

    printf("Ingrese el numero de millas:");
    scanf("%f", &millas);

    kilometros = millas * Millas_Km;

    printf("%.2f millas equivalen a %.3f KM", millas, kilometros);

    return 0;
}
```

Salida del programa

```
C:\Users\ASUS TUF A16\Desktop\TDLP\David-Ruiz_teoria-de-la-programaci-n\Unidad 1\20 ejercicios>e47
Ingrese el numero de millas:13
13.00 millas equivalen a 20.921 KM
C:\Users\ASUS TUF A16\Desktop\TDLP\David-Ruiz_teoria-de-la-programaci-n\Unidad 1\20 ejercicios>e47
Ingrese el numero de millas:76
76.00 millas equivalen a 122.310 KM
```

Conclusiones del algoritmo

Esté algoritmo permite transformar rápidamente una cantidad de millas a kilómetros

Ejercicio 48

Enunciado del algoritmo

En una empresa de transporte, diseñe un algoritmo que lea los tres lados (reales) de un triángulo. Calcule el perímetro con la expresión aritmética (lado1 + lado2 + lado3) y asígnelo a perímetro. Evalúe la expresión lógica (lado1 > 0 AND lado2 > 0 AND lado3 > 0). Muestre por pantalla el perímetro del triángulo. Además del cálculo principal, calcule el resto de una división adicional. Incluya un mensaje de confirmación al finalizar la operación.

Análisis del problema

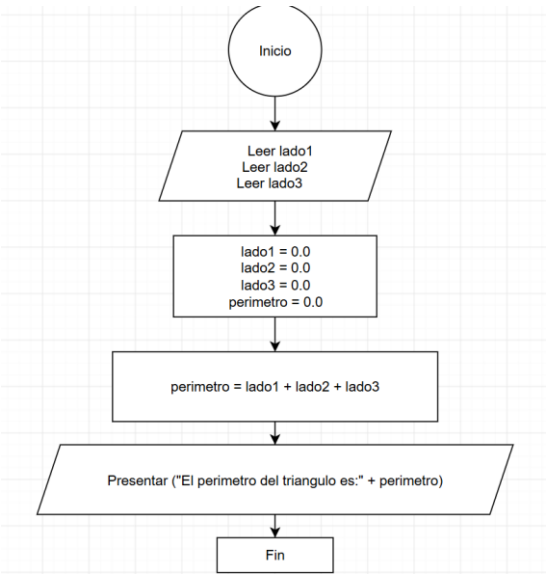
Datos de entrada: Lado 1, 2 y 3 del triángulo y que sean mayores que cero

Proceso: suma de los tres lados para sacar el perímetro resultante

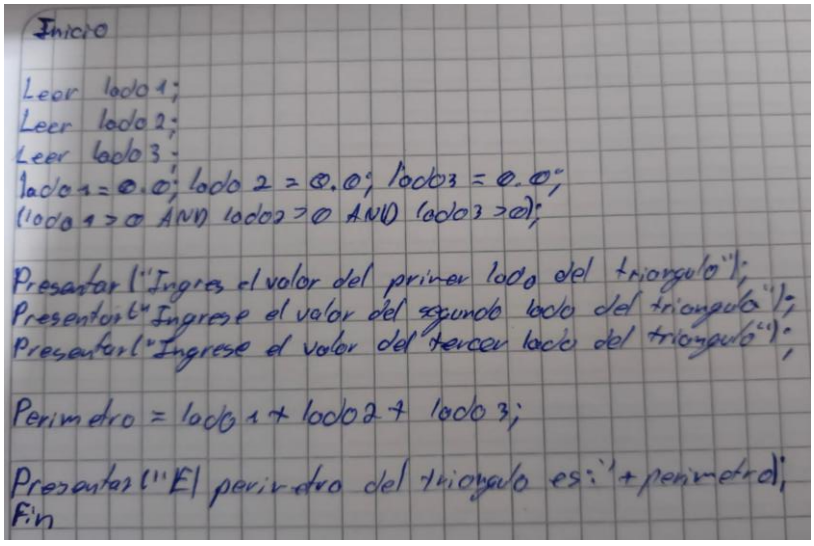
Datos de salida: perímetro calculado tras la suma de los 3 lados

UNIVERSIDAD NACIONAL DE LOJA

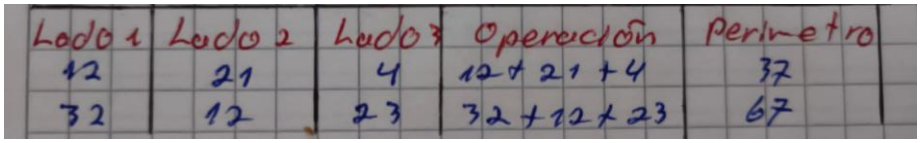
Diagrama de flujo



Pseudocódigo



Prueba de escritorio



Lado 1	Lado 2	Lado 3	Operación	Perimetro
12	21	4	12 + 21 + 4	37
32	12	23	32 + 12 + 23	67

Codificación en lenguaje C

```c
#include <stdio.h>

int main() {
    float lado1 = 0.0, lado2, lado3 = 0.0, perimetro = 0.0;
    (lado1 > 0 && lado2 > 0 && lado3 > 0);

    printf("Ingrese el valor del primer lado del triangulo:");
    scanf("%f", &lado1);

    printf("Ingrese el valor del segundo lado del triangulo:");
    scanf("%f", &lado2);

    printf("Ingrese el valor del tercer lado del triangulo:");
    scanf("%f", &lado3);

    perimetro = lado1 + lado2 + lado3;

    printf("El perimetro del triangulo es: %.2f cm", perimetro);

    return 0;
}
```

Salida del programa

```
C:\Users\ASUS TUF A16\Desktop\TDLP\David-Ruiz_teoria-de-la-programaci-n\Unidad 1\20 ejercicios>e48
Ingrese el valor del primer lado del triangulo:12
Ingrese el valor del segundo lado del triangulo:21
Ingrese el valor del tercer lado del triangulo:4
El perimetro del triangulo es: 37.00
C:\Users\ASUS TUF A16\Desktop\TDLP\David-Ruiz_teoria-de-la-programaci-n\Unidad 1\20 ejercicios>e48
Ingrese el valor del primer lado del triangulo:32
Ingrese el valor del segundo lado del triangulo:12
Ingrese el valor del tercer lado del triangulo:23
El perimetro del triangulo es: 67.00
```

Conclusiones del algoritmo

Este algoritmo nos permite calcular el perímetro de un triángulo fácil y rápidamente

Ejercicio 49

Enunciado del algoritmo

En una zapatería, cree un algoritmo que lea la edad de un perro en años humanos (entero). Use una constante simbólica FACTOR_CANINO = 7. Calcule la edad canina con la expresión aritmética (edad_humana * FACTOR_CANINO) y asígnela a edad_canina. Evalúe la expresión relacional (edad_humana > 0). Muestre por pantalla 'X años humanos equivalen a Y años caninos'. Además, calcule el valor mínimo de todas las variables utilizadas. Agregue validaciones para que no haya resultados negativos.

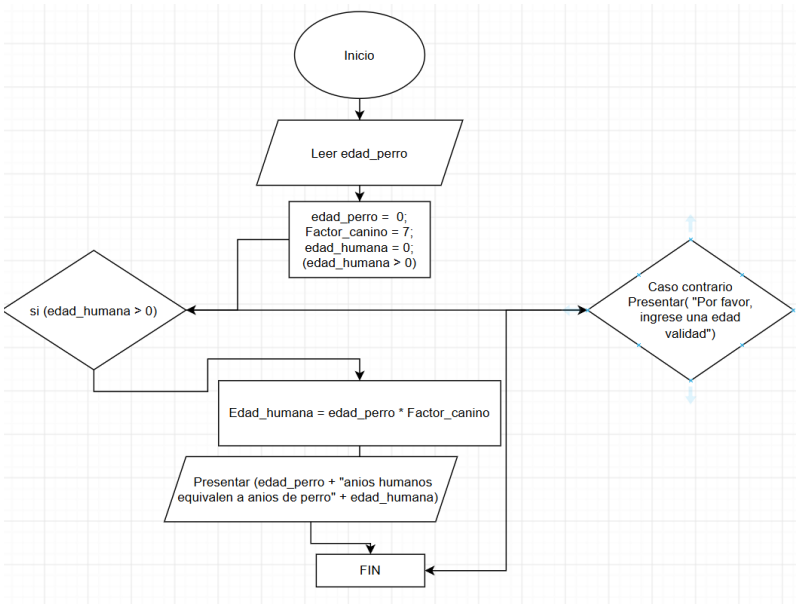
Análisis del problema

Datos de entrada: la edad humana del perro y la evaluación que esta debe ser mayor a cero

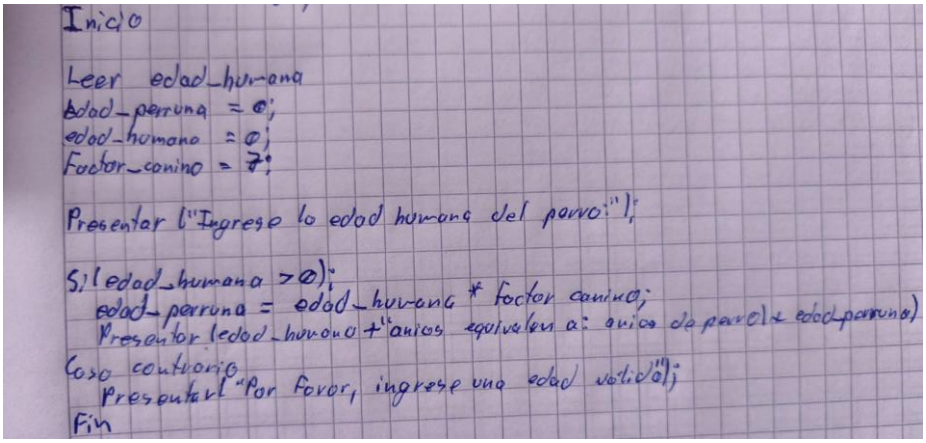
Proceso: Se multiplica la edad humana del perro por el factor canino que es igual a 7, tomando en cuenta la evaluación inicial

Datos de salida: La edad perruna de la edad humana y si no cumple con la evaluación, se dará una advertencia

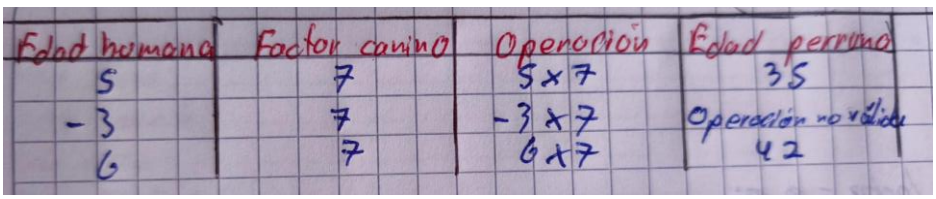
Diagrama de flujo



Pseudocódigo



Prueba de escritorio



Codificación en lenguaje C

```c
#include <stdio.h>

int main(){
    int edad_humana = 0, edad_perruna = 0;
    int factor_canino = 7;

    printf("Ingrese la edad humana del perro:");
    scanf("%d", &edad_humana);

    if (edad_humana > 0) {
        edad_perruna = edad_humana * factor_canino;
        printf("%d anios humanos equivalen a %d anios de perro", edad_humana, edad_perruna);
    } else {
        printf("Por favor, ingrese una edad valida.");
    }

    return 0;
}
```

Salida del programa

```
C:\Users\ASUS TUF A16\Desktop\TDLP\David-Ruiz_teoria-de-la-programaci-n\Unidad 1\20 ejercicios>e49
Ingrese la edad humana del perro:5
5 anios humanos equivalen a 35 anios de perro
C:\Users\ASUS TUF A16\Desktop\TDLP\David-Ruiz_teoria-de-la-programaci-n\Unidad 1\20 ejercicios>e49
Ingrese la edad humana del perro:-3
Por favor, ingrese una edad valida.
C:\Users\ASUS TUF A16\Desktop\TDLP\David-Ruiz_teoria-de-la-programaci-n\Unidad 1\20 ejercicios>e49
Ingrese la edad humana del perro:6
6 anios humanos equivalen a 42 anios de perro
```

Conclusiones del algoritmo

Este algoritmo nos ayuda a verificar rápidamente cual es la edad perruna de nuestra mascota y gracias a la evaluación, no da margen de incoherencias.

Ejercicio 50

Enunciado del algoritmo

En una cooperativa, construya un algoritmo que lea una altura en pies (real). Use una constante simbólica PIE_METRO = 0.3048. Calcule la equivalencia en metros con la expresión aritmética (pies * PIE_METRO) y asígnela a metros. Evalúe la expresión relacional (metros > 0). Muestre por pantalla 'X pies equivalen a Y metros'. Además, determine si el resultado es par o impar (cuando aplique). Incluya mensajes informativos durante la ejecución del algoritmo.

Análisis del problema

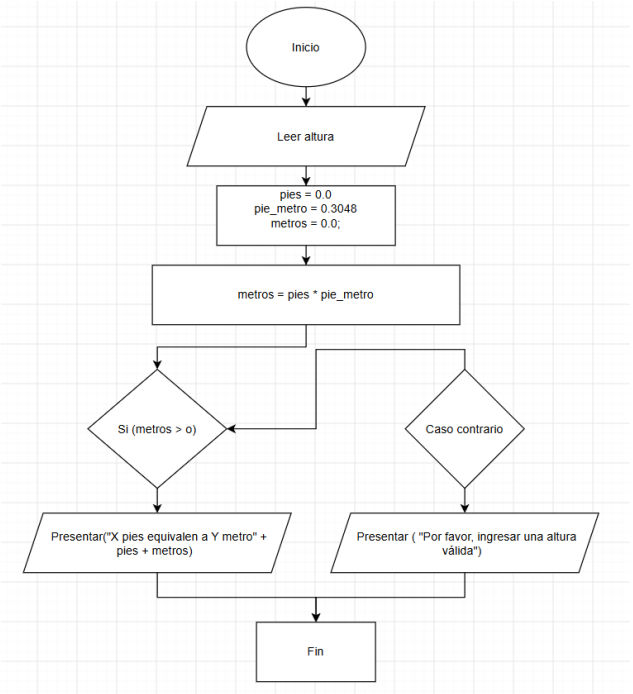
Datos de entrada: Leer cuantos pies se quiere transformar a metros y el valor de transformación de pies a metros

Proceso: Se multiplica el número de pies ingresados por el valor de transformación

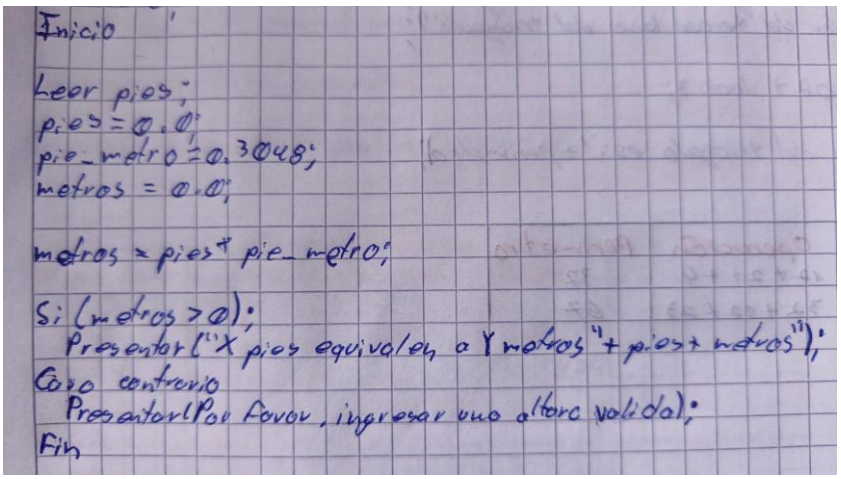
UNIVERSIDAD NACIONAL DE LOJA

Datos de salida: Se hace una evaluación, si el numero sale positivo, se presentar la transformación, casos contrarios, si sale negativo, se presenta una advertencia

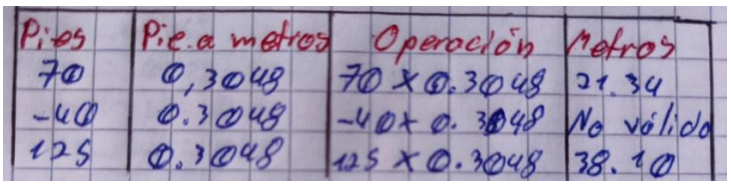
Diagrama de flujo



Pseudocódigo



Prueba de escritorio



Pies	Pie a metros	Operación	Metros
70	0,3048	70 x 0.3048	21.34
-40	0.3048	-40 x 0.3048	No válido
125	0.3048	125 x 0.3048	38.10

Codificación en lenguaje C

```c
#include <stdio.h>
#include <math.h>

int main(){
    float pies = 0.0;
    float pie_metros = 0.3048;
    float metros = 0.0;

    printf("Ingrese la altura en pies:");
    scanf("%f", &pies);

    metros = pies * pie_metros;
    if (metros > 0){
        printf("%.2f pies equivalen a %.2f metros", pies, metros);
    } else {
        printf("Por favor, ingrese una altura valida.");
    }
}
```

Salida del programa

```
C:\Users\ASUS TUF A16\Desktop\TDLP\David-Ruiz_teoria-de-la-programaci-n\Unidad 1\20 ejercicios>e50
Ingrese la altura en pies:70
70.00 pies equivalen a 21.34 metros
C:\Users\ASUS TUF A16\Desktop\TDLP\David-Ruiz_teoria-de-la-programaci-n\Unidad 1\20 ejercicios>e50
Ingrese la altura en pies:-40
Por favor, ingrese una altura valida.
C:\Users\ASUS TUF A16\Desktop\TDLP\David-Ruiz_teoria-de-la-programaci-n\Unidad 1\20 ejercicios>e50
Ingrese la altura en pies:125
125.00 pies equivalen a 38.10 metros
```

Conclusiones del algoritmo

Este algoritmo hace una evaluación en los datos de salida, contrario al algoritmo anterior, que lo hace al momento de ingresar los datos de entrada. Nos ayuda a calcular rápidamente cuantos pies equivalen a cuantos metros

Ejercicio 146

Enunciado del algoritmo

En una universidad, diseñe un algoritmo que lea el peso (real, kg) y la altura (real, m). Use constantes simbolicas BAJO = 18.5, NORMAL = 25.0 y ALTO = 30.0. Calcule el IMC con (peso / (altura * altura)). Use expresiones logicas relacionales para clasificar en bajo peso, normal, sobrepeso u obesidad. Asigne la categoria a una variable cadena. Muestre por pantalla el IMC y la categoria. Adicionalmente, calcule la potencia del resultado elevado a la 4. Incluya una seccion que explique brevemente el propósito del algoritmo.

Análisis del problema

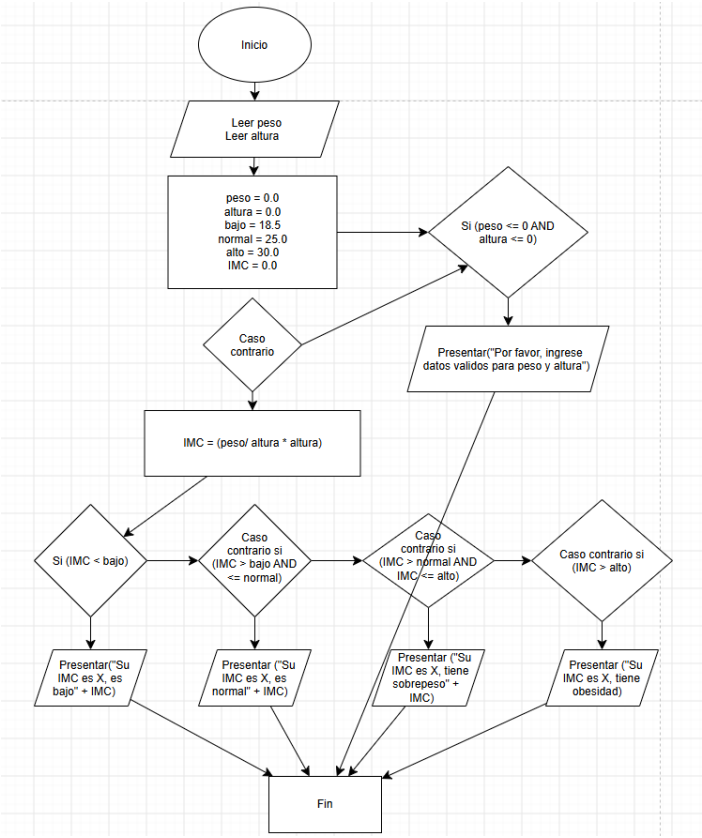
Datos de entrada: peso en kg, altura en m, constantes simbólicas que representar si su IMC es bajo, normal o alto

Proceso: Calcular el IMC dividiendo el peso por la altura elevada a sí misma

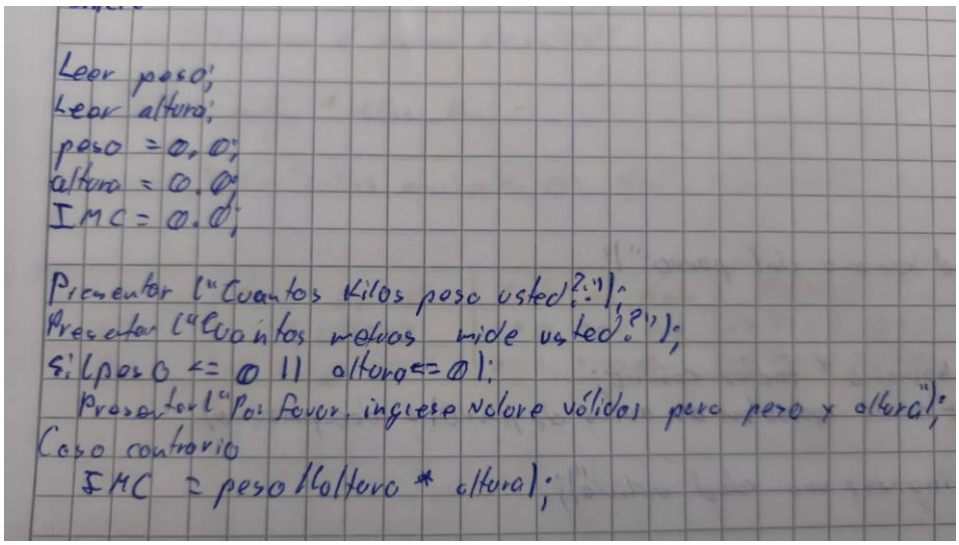
Datos de salida: Presentar si la persona tiene si IMC bajo, normal, elevado o en sobrepeso

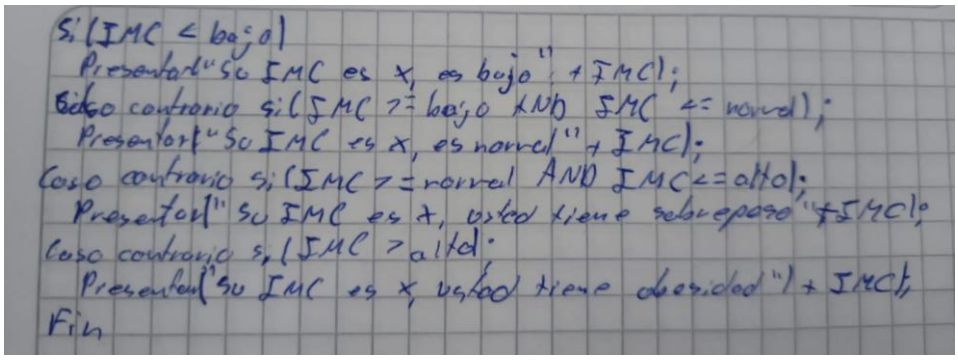
UNIVERSIDAD NACIONAL DE LOJA

Diagrama de flujo

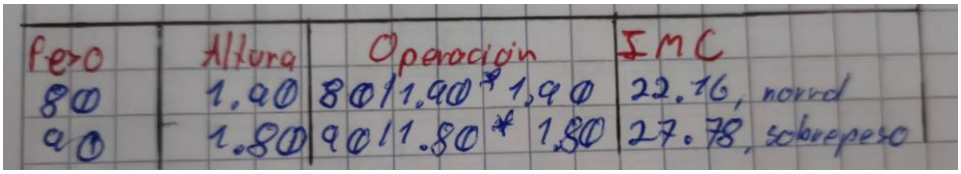


Pseudocódigo





Prueba de escritorio



Peso	Altura	Operacion	IMC
80	1.90	80/1.90 * 1.90	22.16, normal
90	1.80	90/1.80 * 1.80	27.78, sobrepeso

Codificación en lenguaje C

```c
#include <stdio.h>

int main() {
    float peso = 0.0, altura = 0.0, IMC = 0.0;
    float bajo = 18.5, normal = 24.9, alto = 29.9;

    printf("¿Cuantos kilos pesa usted?: ");
    scanf("%f", &peso);

    printf("¿Cuantos metros mide usted?: ");
    scanf("%f", &altura);

    if (peso <= 0 || altura <= 0) {
        printf("Por favor, ingrese valores validos para peso y altura.");
        return 1;
    } else {
        IMC = peso / (altura * altura);
    }

    if(IMC < bajo) {
        printf("Su IMC es %.2f, es bajo.", IMC);
    } else if(IMC >= bajo && IMC <= normal) {
        printf("Su IMC es %.2f, es normal.", IMC);
    } else if(IMC > normal && IMC <= alto) {
        printf("Su IMC es %.2f, usted tiene sobrepeso.", IMC);
    } else if(IMC > alto) {
        printf("Su IMC es %.2f, usted tiene obesidad.", IMC);
    }
    return 0;
}
```

Salida del programa

```
C:\Users\ASUS TUF A16\Desktop\TDLP\David-Ruiz_teoria-de-la-programaci-n\Unidad 1\20 ejercicios>e146
¿Cuantos kilos pesa usted?: 80
¿Cuantos metros mide usted?: 1.90
Su IMC es 22.16, es normal.
C:\Users\ASUS TUF A16\Desktop\TDLP\David-Ruiz_teoria-de-la-programaci-n\Unidad 1\20 ejercicios>e146
¿Cuantos kilos pesa usted?: 90
¿Cuantos metros mide usted?: 1.80
Su IMC es 27.78, usted tiene sobrepeso.
```

Conclusiones del algoritmo

Ejercicio 147

Enunciado del algoritmo

En una tienda de electrónica, cree un algoritmo que lea un número real y por teclado. Use la constante literal 2 como exponente. Calcule la potencia con la

expresión aritmética (y ^ 2) y asígnela a cuadrado. Evalúe la expresión relacional (cuadrado >= 0). Muestre por pantalla 'El cuadrado de X es Y'. Adicionalmente, calcule la diferencia entre el resultado y el promedio. Muestre al inicio una lista de las variables que se utilizaran.

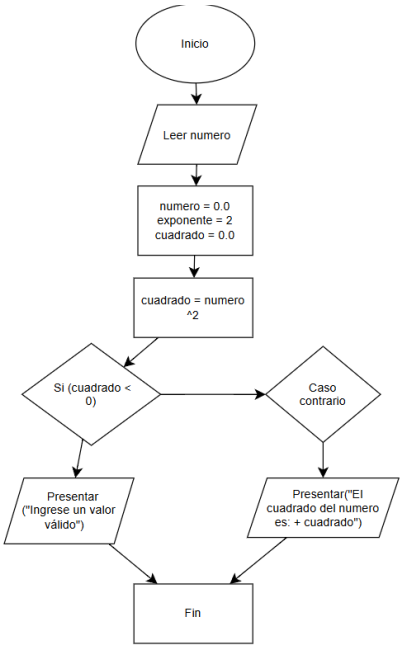
Análisis del problema

Datos de entrada: Leer un número real, la constante exponente al cuadrado y la evaluación de números negativos

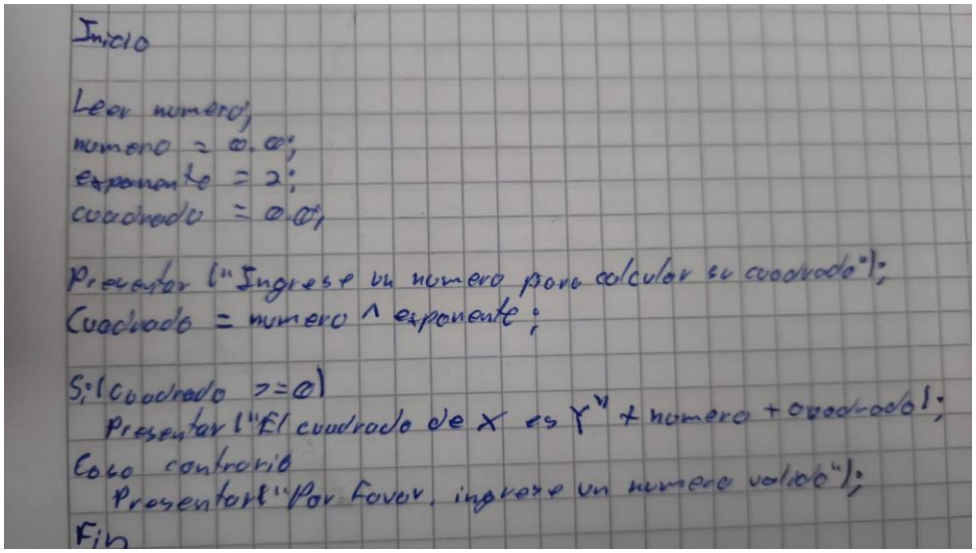
Proceso: Calcular la potencia al cuadrado del número real ingresado

Datos salida: El numero ingresado elevado al cuadrado, más la diferencia entre el resultado y el promedio

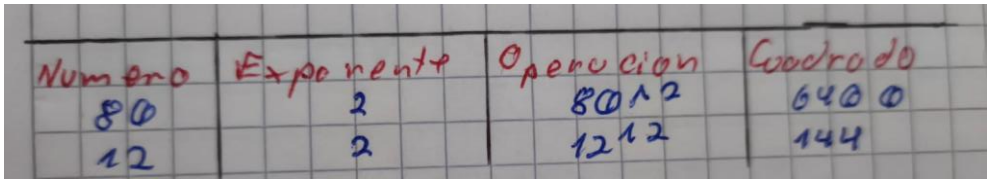
Diagrama de flujo



Pseudocódigo



Prueba de escritorio



Numero	Exponente	Operacion	Cuadrado
80	2	80^2	6400
12	2	12^2	144

Codificación en lenguaje C

```c
#include <stdio.h>
#include <math.h>

int main(){
    float numero = 0.0, cuadrado = 0.0;
    int exponente = 2;

    printf("Ingrese un numero para calcular su cuadrado: ");
    scanf("%f", &numero);

    cuadrado = pow(numero, exponente);

    if (cuadrado >= 0) {
        printf("El cuadrado de %.2f es %.2f", numero, cuadrado);
    } else {
        printf("Por favor, ingrese un numero valido.");
    }
    return 0;
}
```

Salida del programa

```
C:\Users\ASUS TUF A16\Desktop\TDLP\David-Ruiz_teoria-de-la-programaci-n\Unidad 1\20 ejercicios>e147
Ingrese un numero para calcular su cuadrado: 80
El cuadrado de 80.00 es 6400.00
C:\Users\ASUS TUF A16\Desktop\TDLP\David-Ruiz_teoria-de-la-programaci-n\Unidad 1\20 ejercicios>e147
Ingrese un numero para calcular su cuadrado: 12
El cuadrado de 12.00 es 144.00
```

Conclusiones del algoritmo

Ejercicio 148

UNIVERSIDAD NACIONAL DE LOJA

Enunciado del algoritmo

En un gimnasio, construya un algoritmo que lea cinco notas (reales). Use la constante literal 3 como divisor. Identifique la mayor y la menor usando expresiones relacionales. Calcule el promedio de las tres notas intermedias con la expresión aritmética ((suma - mayor - menor) / 3). Asígnelo a promedio. Muestre por pantalla el promedio. Adicionalmente, calcule el porcentaje del resultado respecto a un valor de referencia. Justifique el tipo de dato simple elegido para cada variable.

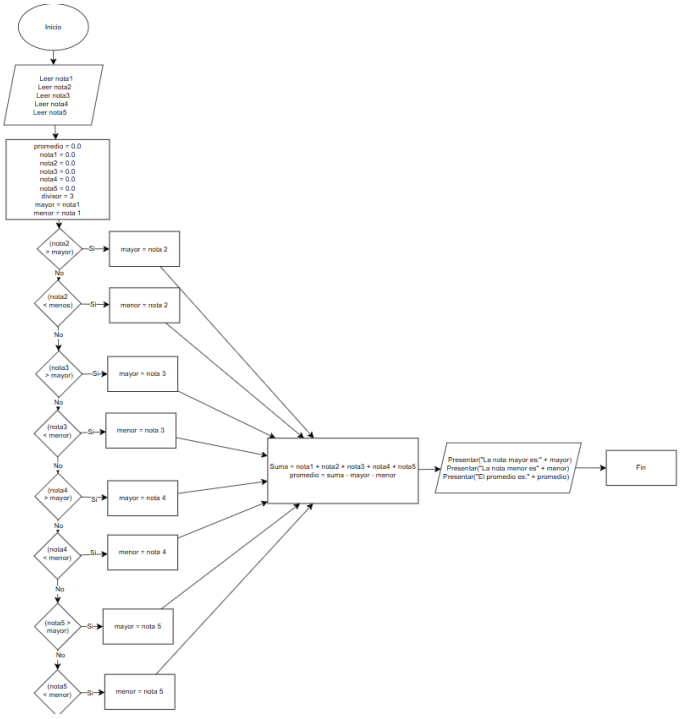
Análisis del problema

Datos de entrada: Cinco notas y constante divisora tres

Proceso: Primero, identificar la nota menor y la mayor de las cinco, después promediar los tres restantes, sumándolas y dividiéndolas para tres

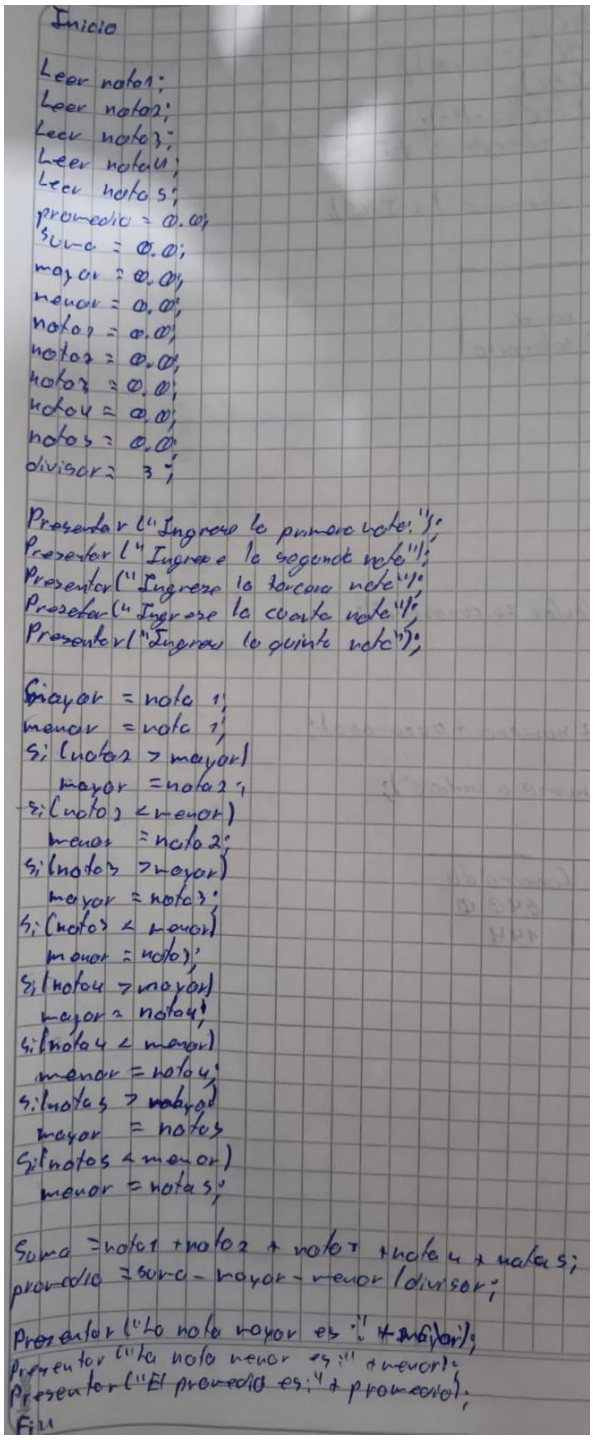
Datos de salida: Cual es la nota mayor, cual es la nota menor, y cuál es el promedio de las tres notas intermedias

Diagrama de flujo



Pseudocódigo

UNIVERSIDAD NACIONAL DE LOJA



```
Inicio

Leer nota1;
Leer nota2;
Leer nota3;
Leer nota4;
Leer nota5;
promedio = 0.0;
Suma = 0.0;
mayor = 0.0;
menor = 0.0;
nota1 = 0.0;
nota2 = 0.0;
nota3 = 0.0;
nota4 = 0.0;
nota5 = 0.0;
divisor = 3;

Presentar("Ingrese la primera nota:");
Presentar("Ingrese la segunda nota");
Presentar("Ingrese la tercera nota");
Presentar("Ingrese la cuarta nota");
Presentar("Ingrese la quinta nota");

Mayor = nota 1;
menor = nota 1;
Si(nota2 > mayor)
   mayor = nota2;
Si(nota2 < menor)
   menor = nota2;
Si(nota3 > mayor)
   mayor = nota3;
Si(nota3 < menor)
   menor = nota3;
Si(nota4 > mayor)
   mayor = nota4;
Si(nota4 < menor)
   menor = nota4;
Si(nota5 > mayor)
   mayor = nota5
Si(nota5 < menor)
   menor = nota5;

Suma = nota1 + nota2 + nota3 + nota4 + nota5;
promedio = suma - mayor - menor / divisor;

Presentar("La nota mayor es:" + mayor);
Presentar("La nota menor es:" + menor);
Presentar("El promedio es:" + promedio);
Fin
```

Prueba de escritorio

Nota1	Nota2	Nota3	Nota4	Nota5	Mayor	Menor	Operacion	Promedio
7	6	8	8	9	9	6	38-6-9/3	7.67
45	70	30.5	92	57	92	30.5	303,5-92-30.5/3	60.33

Codificación en lenguaje C

```c
#include <stdio.h>

int main(){
    float nota1 = 0.0, nota2 = 0.0, nota3 = 0.0, nota4 = 0.0, nota5 = 0.0;
    float promedio = 0.0, suma = 0.0, mayor = 0.0, menor = 0.0;
    int divisor = 3;

    printf("Ingrese la primera nota: ");
    scanf("%f", &nota1);
    printf("Ingrese la segunda nota: ");
    scanf("%f", &nota2);
    printf("Ingrese la tercera nota: ");
    scanf("%f", &nota3);
    printf("Ingrese la cuarta nota: ");
    scanf("%f", &nota4);
    printf("Ingrese la quinta nota: ");
    scanf("%f", &nota5);

    mayor = nota1;
    menor = nota1;

    if (nota2 > mayor) {
        mayor = nota2;
    }
    if (nota2 < menor) {
        menor = nota2;
    }

    if (nota3 > mayor) {
        mayor = nota3;
    }
    if (nota3 < menor) {
        menor = nota3;
    }

    if (nota4 > mayor) {
        mayor = nota4;
    }
    if (nota4 < menor) {
        menor = nota4;
    }

    if (nota5 > mayor) {
        mayor = nota5;
    }
    if (nota5 < menor) {
        menor = nota5;
    }

    suma = nota1 + nota2 + nota3 + nota4 + nota5;
    promedio = (suma - mayor - menor) / divisor;

    printf("La nota mayor es: %.2f\n", mayor);
    printf("La nota menor es: %.2f\n", menor);
    printf("El promedio es: %.2f\n", promedio);
}
```

Salida del programa

```
C:\Users\ASUS TUF A16\Desktop\TDLP\David-Ruiz_teoria-de-la-programaci-n\Unidad 1\20 ejercicios>e148
Ingrese la primera nota: 7
Ingrese la segunda nota: 6
Ingrese la tercera nota: 8
Ingrese la cuarta nota: 8
Ingrese la quinta nota: 9
La nota mayor es: 9.00
La nota menor es: 6.00
El promedio es: 7.67

C:\Users\ASUS TUF A16\Desktop\TDLP\David-Ruiz_teoria-de-la-programaci-n\Unidad 1\20 ejercicios>e148
Ingrese la primera nota: 45
Ingrese la segunda nota: 79
Ingrese la tercera nota: 30.5
Ingrese la cuarta nota: 92
Ingrese la quinta nota: 57
La nota mayor es: 92.00
La nota menor es: 30.50
El promedio es: 60.33
```

Conclusiones del algoritmo

El algoritmo determina correctamente cual es la nota menor, la mayor, y el promedio que se genera entre las 3 notas intermedias y evalúa que todas sean positivas

Ejercicio 149

Enunciado del algoritmo

En una florería, diseñe un algoritmo que lea una temperatura en grados Fahrenheit (real). Use constantes simbólicas FACTOR = 0.5556 y OFFSET_K = 273.15. Calcule la temperatura en Kelvin con la expresión aritmética ((fahrenheit - 32) * FACTOR + OFFSET_K) y asígnela a kelvin. Evalúe la expresión relacional (kelvin >= 0). Muestre por pantalla 'X F equivalen a Y K'. Adicionalmente, redondee el resultado a 2 decimales. Agregue una validación para asegurar que los datos sean positivos.

Análisis del problema

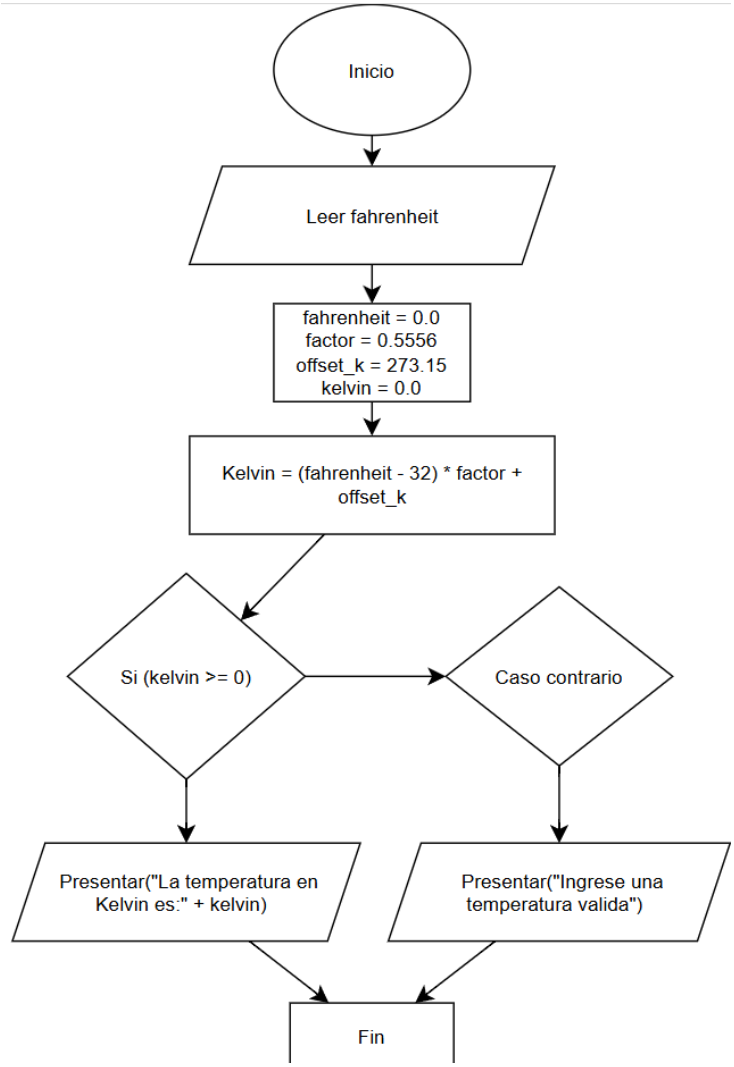
Datos de entrada: leer los grados Fahrenheit que se van a transformar a kelvin, las constantes como el factor de transformación a de f a kelvin

Proceso: a los grados ingresados por el usuario se le resta 32, para después multiplicar por el factor y sumar el factor de conversión a kelvin

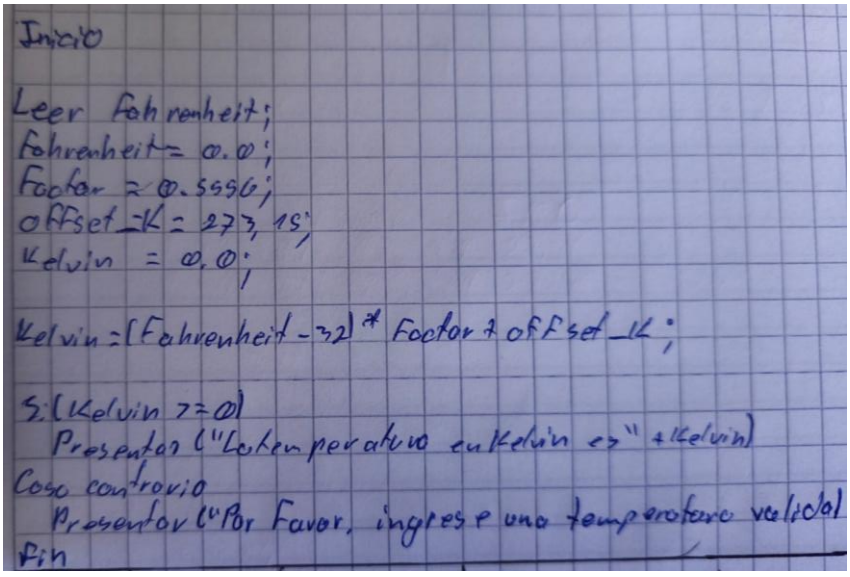
Datos de salida: Cual es el resultado de la transformación de Fahrenheit a kelvin

Diagrama de flujo

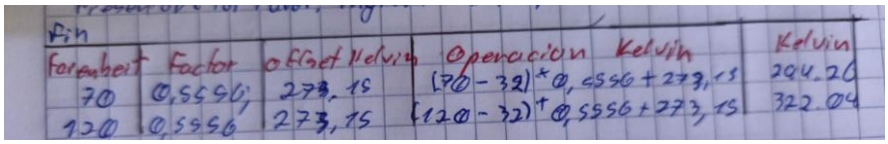
UNIVERSIDAD NACIONAL DE LOJA



Pseudocódigo



Prueba de escritorio



Farenheit	Factor	offset Kelvin	Operacion	Kelvin	Kelvin
70	0,5556	273,15	(70 - 32) * 0,5556 + 273,15		294.26
120	0,5556	273,15	(120 - 32) * 0,5556 + 273,15		322.04

Codificación en lenguaje C

```c
#include <stdio.h>

int main(){
    float farenheit = 0.0, kelvin = 0.0;
    float factor = 0.5556, offset_k = 273.15;

    printf("Ingrese la temperatura en Farenheit: ");
    scanf("%f", &farenheit);

    kelvin = (farenheit - 32) * factor + offset_k;

    if(kelvin >= 0){
        printf("La temperatura en Kelvin es: %.2f\n", kelvin);
    } else {
        printf("Por favor, ingrese una temperatura valida.");
    }
    return 0;
}
```

Salida del programa

```
C:\Users\ASUS TUF A16\Desktop\TDLP\David-Ruiz_teoria-de-la-programaci-n\Unidad 1\20 ejercicios>e149
Ingrese la temperatura en Farenheit: 70
La temperatura en Kelvin es: 294.26

C:\Users\ASUS TUF A16\Desktop\TDLP\David-Ruiz_teoria-de-la-programaci-n\Unidad 1\20 ejercicios>e149
Ingrese la temperatura en Farenheit: 120
La temperatura en Kelvin es: 322.04
```

Conclusiones del algoritmo

El algoritmo evalúa correctamente si la cantidad de grados es positiva para presentar la transformación de Fahrenheit a kelvin

Ejercicio 150

Enunciado del algoritmo

En un hotel, cree un algoritmo que lea un numero entero y por teclado. Use la constante literal 5. Evalúe la expresión lógica (y % 5 == 0 AND y != 0). Asigne el resultado booleano a es_multiplo_5. Muestre por pantalla si el número es múltiplo de 5 o no. Identifique el tipo de dato de cada variable. Adicionalmente, calcule el resultado divido entre una constante simbólica. Verifique que las constantes estén declaradas al inicio del algoritmo.

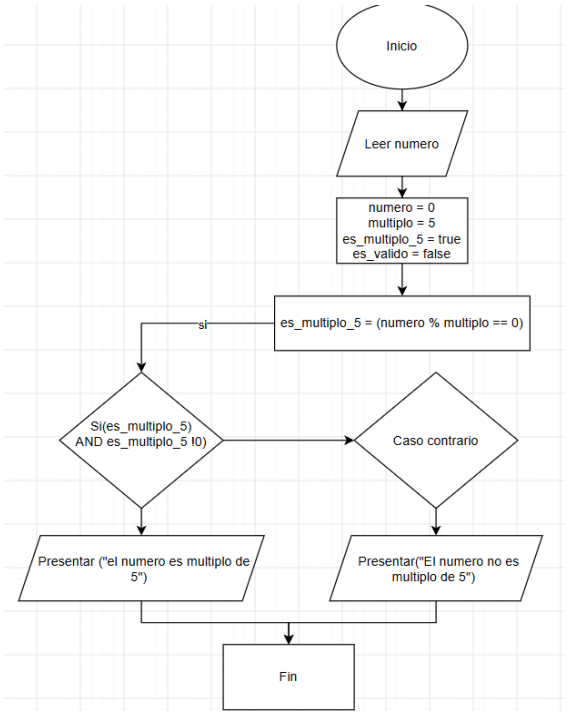
Análisis del problema

Datos de entrada: Leer número entero, la contante 5 para saber si es múltiplo

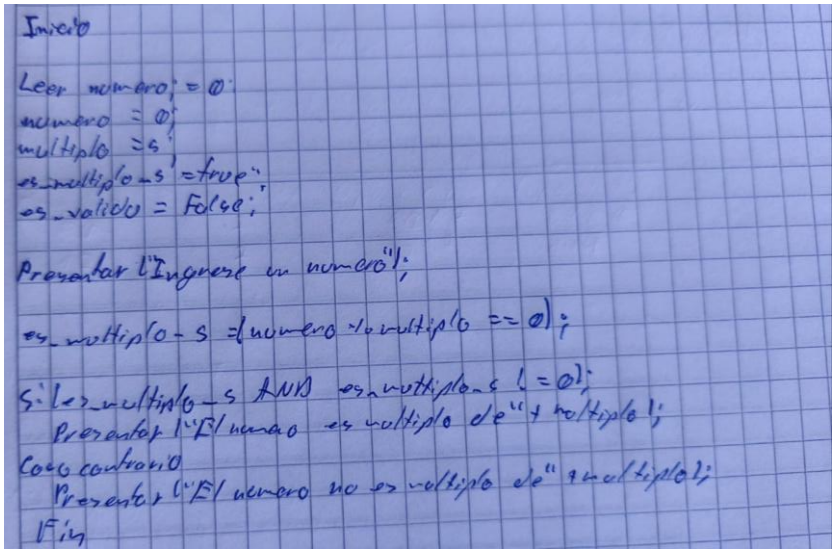
Proceso: el algoritmo debe leer que el numero ingresado sea múltiplo de 5,

Diagrama de flujo

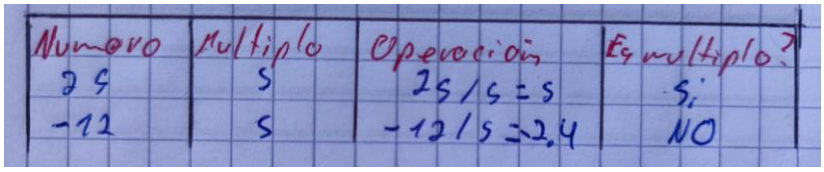
UNIVERSIDAD NACIONAL DE LOJA



Pseudocódigo



Prueba de escritorio



Numero	Multiplo	Operacion	Es multiplo?
25	5	25/5=5	Si
-12	5	-12/5=2.4	NO

Codificación en lenguaje C

```c
#include <stdio.h>

int main(){
    int numero = 0;
    int multiplo = 5;
    bool es_multiplo_5 = true;
    bool esValido = false;

    printf("Ingrese un numero: ");
    scanf("%d", &numero);
    es_multiplo_5 = (numero % multiplo == 0);

    if(es_multiplo_5 && es_multiplo_5 != 0){
        printf("El numero es multiplo de %d", multiplo);
    }else{
        printf("El numero no es multiplo de %d", multiplo);
    }
    return 0;
}
```

Salida del programa

```
C:\Users\ASUS TUF A16\Desktop\TDLP\David-Ruiz_teoria-de-la-programaci-n\Unidad 1\20 ejercicios>150
Ingrese un numero: 25
El numero es multiplo de 5
C:\Users\ASUS TUF A16\Desktop\TDLP\David-Ruiz_teoria-de-la-programaci-n\Unidad 1\20 ejercicios>150
Ingrese un numero: -12
El numero no es multiplo de 5
```

Conclusiones del algoritmo

Este algoritmo verifica correctamente si el numero ingresado es múltiplo de 5, sea negativo o positivo

Ejercicio 246

Enunciado del algoritmo

En una tienda de ropa, construya un algoritmo que lea las bases B y b (reales) y la altura h (real) de un trapecio. Use la constante literal 2. Calcule el área con la expresión aritmética ((B + b) * h / 2) y asígnela a área. Evalúe la expresión lógica (B > b AND h > 0). Muestre por pantalla el área del trapecio. Además del cálculo principal, multiplique el resultado por 1.5 como bono. Incluya una variable auxiliar para llevar un contador de operaciones.

Análisis del problema

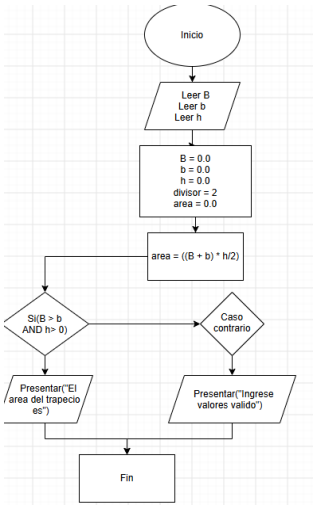
Datos de entrada: Leer las bases y altura de un trapecio, más la contante 2 para dividir y encontrar el área

Proceso: Sumar las bases, multiplicarla por la altura y dividir para 2, para encontrar el área del trapecio

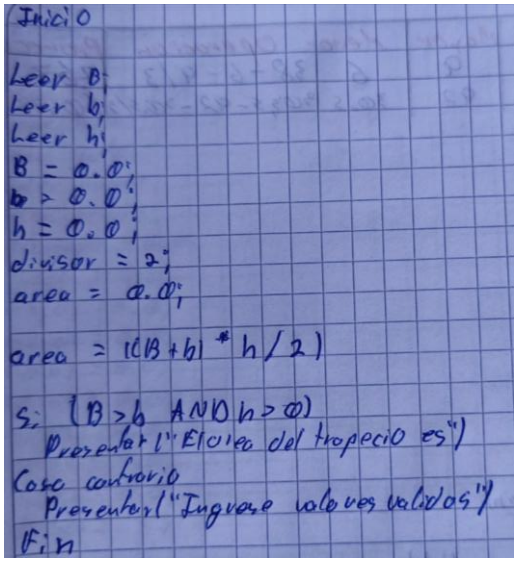
Datos de salida: evaluar si el resultado es correcto y presentar el área del trapecio

Diagrama de flujo

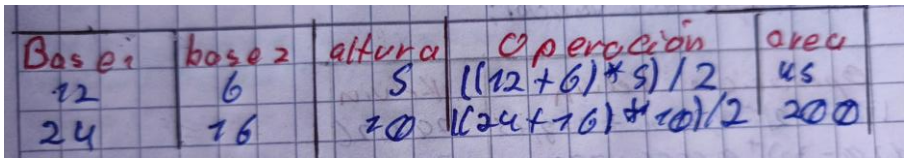
UNIVERSIDAD NACIONAL DE LOJA



Pseudocódigo



Prueba de escritorio



Base 1	base 2	altura	operación	area
12	6	5	((12+6)*5)/2	45
24	16	10	((24+16)*10)/2	200

Codificación en lenguaje C

```c
#include <stdio.h>

int main (){
    float B = 0.0, b = 0.0, h = 0.0, area = 0.0;
    int divisor = 2;

    printf("Ingrese la base mayor: ");
    scanf("%f", &B);
    printf("Ingrese la base menor: ");
    scanf("%f", &b);
    printf("Ingrese la altura: ");
    scanf("%f", &h);

    if (B > b && h > 0) {
        area = ((B + b) * h) / divisor;
        printf("El area del trapecio es: %.2f", area);
    } else {
        printf("Ingrese valores validos.");
    }

    return 0;
}
```

Salida del programa

```
C:\Users\ASUS TUF A16\Desktop\TDLP\David-Ruiz_teoria-de-la-programaci-n\Unidad 1\20 ejercicios>e246
Ingrese la base mayor: 12
Ingrese la base menor: 6
Ingrese la altura: 5
El area del trapecio es: 45.00
C:\Users\ASUS TUF A16\Desktop\TDLP\David-Ruiz_teoria-de-la-programaci-n\Unidad 1\20 ejercicios>e246
Ingrese la base mayor: 24
Ingrese la base menor: 16
Ingrese la altura: 10
El area del trapecio es: 200.00
```

Conclusiones del algoritmo

El algoritmo permite correctamente calcular el área de un trapecio, sin dar resultados negativos

Ejercicio 247

Enunciado del algoritmo

En un taller mecánico, diseñe un algoritmo que lea un peso en kilogramos (real). Use una constante simbólica FACTOR = 2.20462. Calcule la equivalencia en libras con la expresión aritmética (kg * FACTOR) y asígnela a libras. Evalúe la expresión relacional (libras > 0). Muestre por pantalla 'X kg equivalen a Y libras'. Además del cálculo principal, calcule el resultado elevado al cubo. Asegúrese de que las expresiones lógicas estén bien patentizadas.

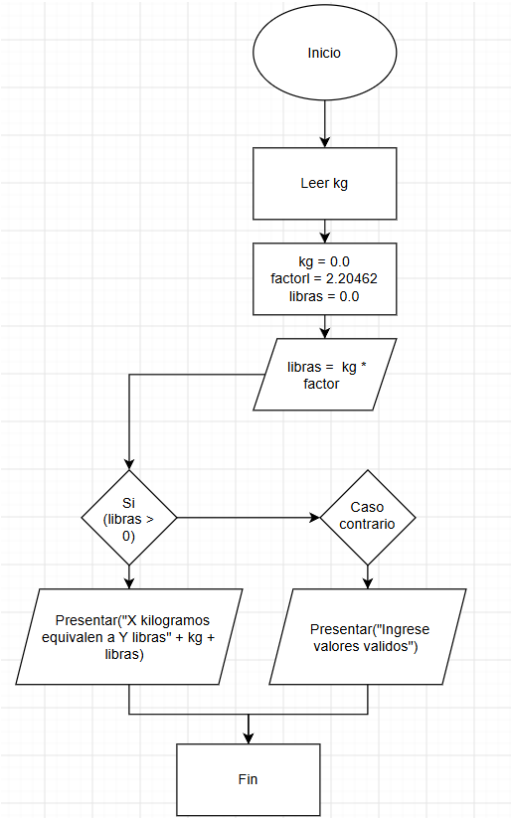
Análisis del problema

Datos de entrada: Leer los kg que se quieren transformar a libras, la contante de transformación de kg a libras

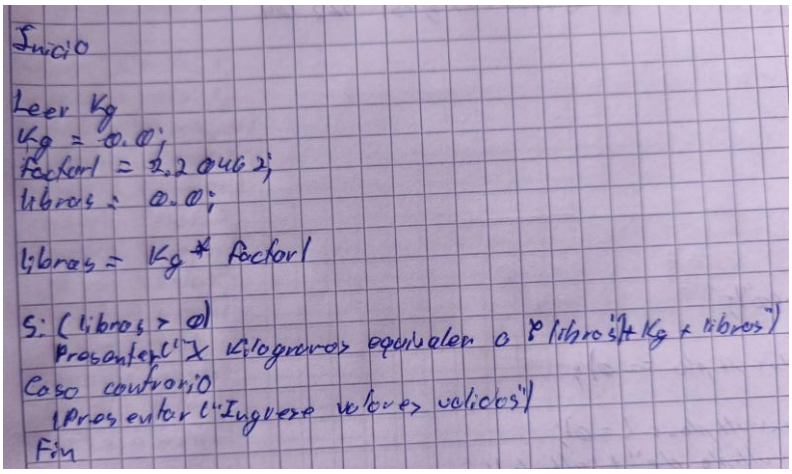
Proceso: Multiplicar los kg ingresados por la contante de transformación a libras, y evaluar si los resultados son positivos

Datos de salida: Los kg transformados a libras, y si la evaluación es negativa, advertencia de ingresar bien los datos

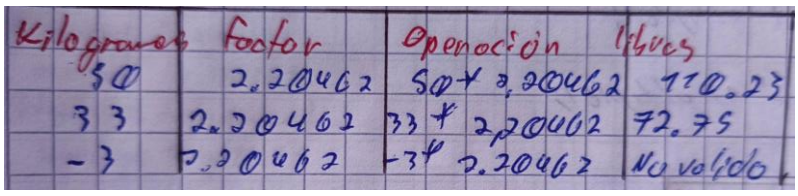
Diagrama de flujo



Pseudocódigo



Prueba de escritorio



Kilogramos	factor	operación	libras
50	2.20462	50 * 2.20462	110.23
33	2.20462	33 * 2.20462	72.75
-3	2.20462	-3 * 2.20462	No valido

Codificación en lenguaje C

```c
#include <stdio.h>

int main(){
    float kg = 0, libras = 0;
    float factorl = 2.20462;

    printf("Ingrese el peso en kilogramos: ");
    scanf("%f", &kg);

    libras = kg * factorl;

    if(libras > 0){
        printf("%.2f kilogramos equivalen a %.2f libras", kg, libras);
    } else {
        printf("Por favor, ingrese un peso valido.");
    }
    return 0;
}
```

Salida del programa

```
C:\Users\ASUS TUF A16\Desktop\TDLP\David-Ruiz_teoria-de-la-programaci-n\Unidad 1\20 ejercicios>e247
Ingrese el peso en kilogramos: 33
33.00 kilogramos equivalen a 72.75 libras
C:\Users\ASUS TUF A16\Desktop\TDLP\David-Ruiz_teoria-de-la-programaci-n\Unidad 1\20 ejercicios>e247
Ingrese el peso en kilogramos: -3
Por favor, ingrese un peso valido.
C:\Users\ASUS TUF A16\Desktop\TDLP\David-Ruiz_teoria-de-la-programaci-n\Unidad 1\20 ejercicios>e247
Ingrese el peso en kilogramos: 50
50.00 kilogramos equivalen a 110.23 libras
```

Conclusiones del algoritmo

El algoritmo cumple con la conversión de kilogramos a libras, evaluando correctamente los valores positivos

Ejercicio 248

Enunciado del algoritmo

En una librería, cree un algoritmo que lea el lado (real) de un triángulo equilátero. Use la constante literal 3. Calcule el perímetro con la expresión aritmética (3 * lado) y asígnelo a perímetro. Evalúe la expresión relacional (lado > 0). Muestre por pantalla el perímetro del triángulo. Además del cálculo principal, calcule el producto del resultado por PI. Asegúrese de inicializar todas las variables antes de usarlas.

Análisis del problema

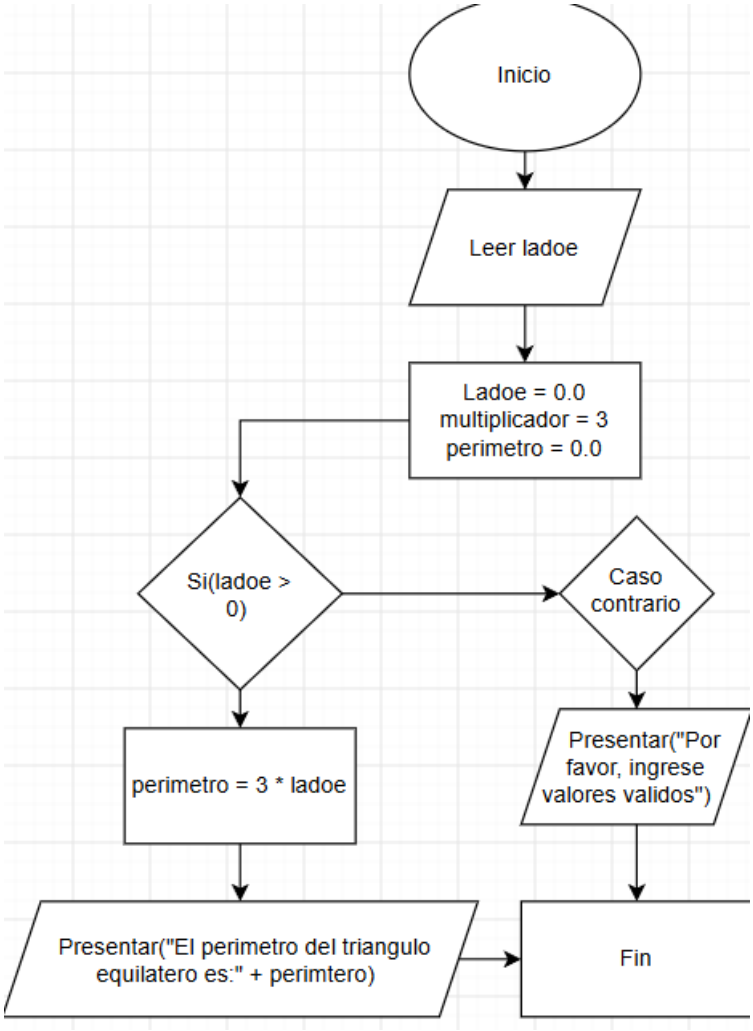
Datos de entrada: Leer el tamaño del lado del triángulo equilátero, más la contante de multiplicación 3

Proceso: Evaluar si el valor del lado ingresado es positivo y multiplicarlo por 3

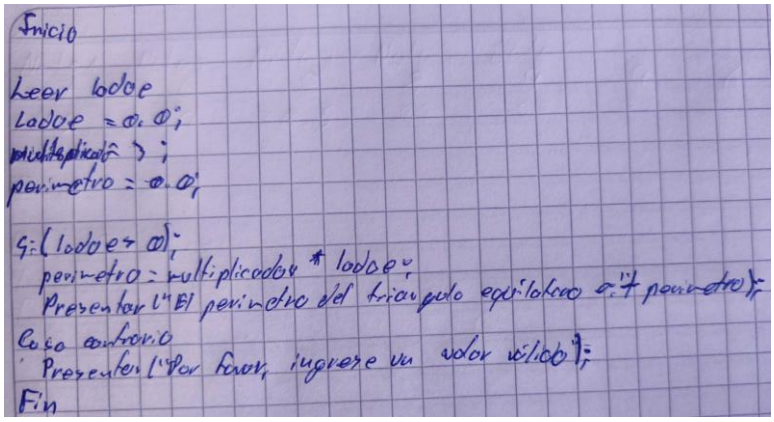
Datos de salida: Si la evaluación es correcta, se presenta el perímetro, si no, se da una advertencia

UNIVERSIDAD NACIONAL DE LOJA

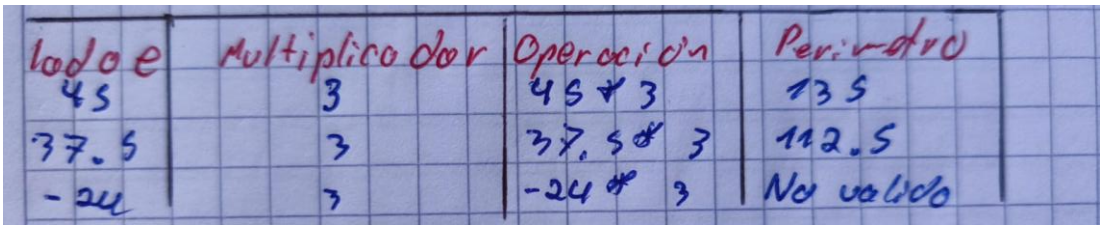
Diagrama de flujo



Pseudocódigo



Prueba de escritorio



ladoe	Multiplicador	Operación	Perimetro
45	3	45 * 3	135
37.5	3	37.5 * 3	112.5
-24	3	-24 * 3	No valido

Codificación en lenguaje C

```c
#include <stdio.h>

int main(){
    float ladoe = 0.0, perimetro = 0.0;
    int multiplicador = 3;

    printf("Ingrese el valor del lado del triangulo: ");
    scanf("%f", &ladoe);

    if(ladoe > 0){
        perimetro = ladoe * multiplicador;
        printf("El perimetro del triangulo equilatero es: %.2f", perimetro);
    } else {
        printf("Por favor, ingrese un valor valido.");
    }
    return 0;
}
```

Salida del programa

```
C:\Users\ASUS TUF A16\Desktop\TDLP\David-Ruiz_teoria-de-la-programaci-n\Unidad 1\20 ejercicios>e248
Ingrese el valor del lado del triangulo: 45
El perimetro del triangulo equilatero es: 135.00
C:\Users\ASUS TUF A16\Desktop\TDLP\David-Ruiz_teoria-de-la-programaci-n\Unidad 1\20 ejercicios>e248
Ingrese el valor del lado del triangulo: 37.5
El perimetro del triangulo equilatero es: 112.50
C:\Users\ASUS TUF A16\Desktop\TDLP\David-Ruiz_teoria-de-la-programaci-n\Unidad 1\20 ejercicios>e248
Ingrese el valor del lado del triangulo: -24
Por favor, ingrese un valor valido.
```

Conclusiones del algoritmo

El algoritmo calcula correctamente el perímetro de un triángulo equilátero después de evaluar los valores positivos

Ejercicio 249

Enunciado del algoritmo

En una empresa de transporte, construya un algoritmo que lea un numero entero n por teclado. Use constantes literales 7 y 11. Evalúe las expresiones lógicas (n % 7 == 0) y (n % 11 == 0). Asigne el resultado de combinarlas con AND a una variable es_divisible_7_11. Muestre por pantalla si el número es divisible por 7 y por 11. Además, calcule el cuadrado del resultado principal. Agregue al final un resumen de los datos ingresados.

UNIVERSIDAD NACIONAL DE LOJA

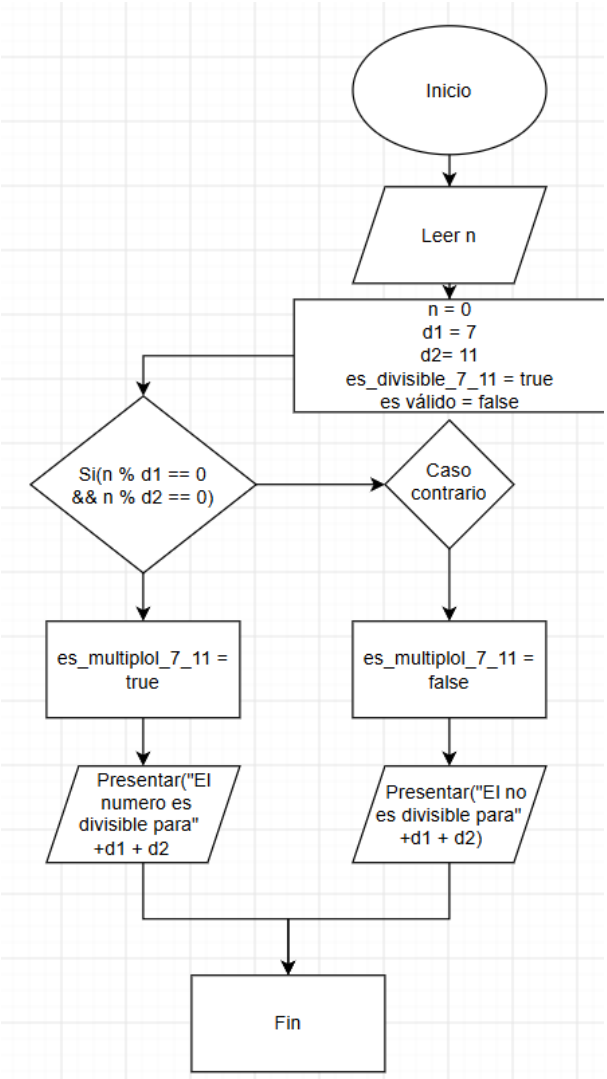
Análisis del problema

Datos de entrada: Leer un numero entero y las constantes 7, 11 para determinar si son divisibles para los dos

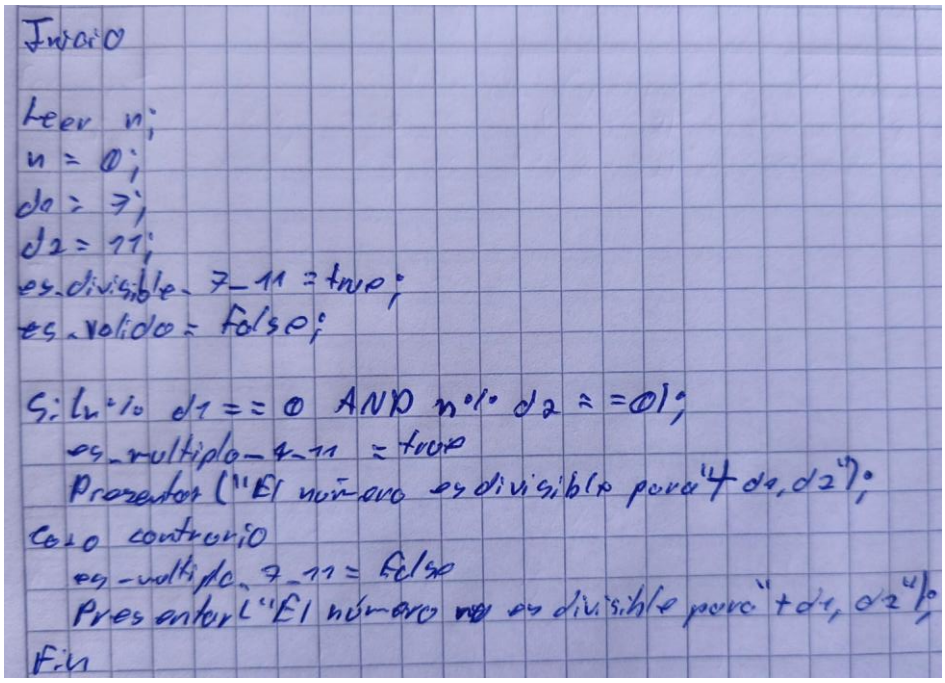
Proceso: Evaluar si el numero ingresado en divisible para 7 y 11

Datos de salida: Presentar si el numero ingresado es divisible para 7 y 11

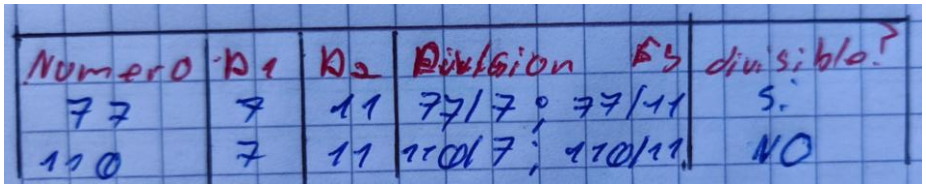
Diagrama de flujo



Pseudocódigo



Prueba de escritorio



Numero	D1	D2	Division	Es divisible?
77	7	11	77/7 ; 77/11	S.
110	7	11	110/7 ; 110/11	NO

Codificación en lenguaje C

```c
#include <stdio.h>

int main(){
    int n = 0;
    int d1 = 7;
    int d2 = 11;
    bool es_multiplo_7_11 = true;
    bool esValido = false;

    printf("Ingrese un numero: ");
    scanf("%d", &n);

    if(n % d1 == 0 && n % d2 == 0){
        es_multiplo_7_11 = true;
        printf("El numero es multiplo de %d y %d", d1, d2);
    } else {
        es_multiplo_7_11 = false;
        printf("El numero no es multiplo de %d y %d", d1, d2);
    }
    return 0;
}
```

Salida del programa

```
C:\Users\ASUS TUF A16\Desktop\TDLP\David-Ruiz_teoria-de-la-programaci-n\Unidad 1\20 ejercicios>e249
Ingrese un numero: 77
El numero es multiplo de 7 y 11
C:\Users\ASUS TUF A16\Desktop\TDLP\David-Ruiz_teoria-de-la-programaci-n\Unidad 1\20 ejercicios>e249
Ingrese un numero: 110
El numero no es multiplo de 7 y 11
```

Conclusiones del algoritmo

El algoritmo evalúa correctamente si el numero ingresado es divisible para 7 y 11

Ejercicio 250

Enunciado del algoritmo

En una zapatería, diseñe un algoritmo que lea una presión en hectopascales (real). Use una constante simbólica FACTOR = 0.750062. Calcule la equivalencia en mmHg con la expresión aritmética (hPa * FACTOR) y asígnela a mmHg. Evalúe la expresión relacional (mmHg > 0). Muestre por pantalla 'X hPa equivalen a Y mmHg'. Además, calcule el valor absoluto de la diferencia entre dos resultados. Identifique explícitamente los datos de entrada y la información de salida.

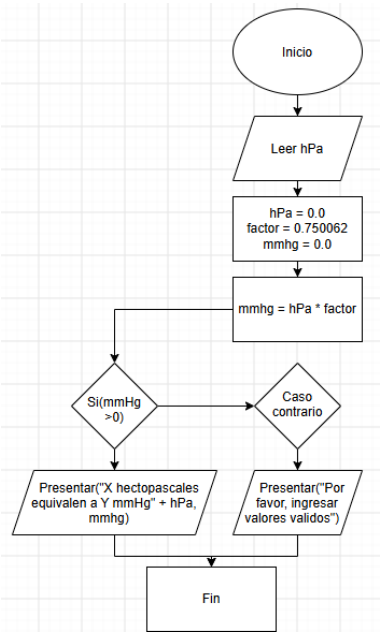
Análisis del problema

Datos de entrada: Leer hectopascales, la constante de transformación a mmHg

Proceso: Multiplicar los hectopascales ingresados por el factor de transformación y evaluar si son positivos

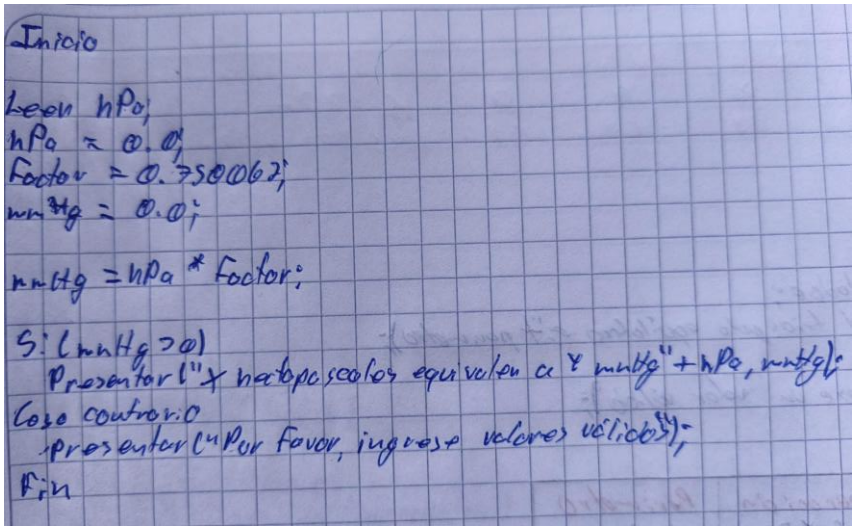
Datos de salida: Si la evaluación es correcta, presentar los mmHg equivalentes, caso contrario, dará una advertencia

Diagrama de flujo

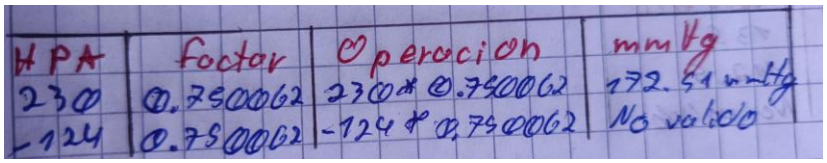


UNIVERSIDAD NACIONAL DE LOJA

Pseudocódigo



Prueba de escritorio



Codificación en lenguaje C

```c
#include <stdio.h>

int main(){
    float hPa = 0.0, mmHg = 0.0;
    float factor = 0.750062;

    printf("Ingrese la presion en hPa: ");
    scanf("%f", &hPa);

    mmHg = hPa * factor;

    if(mmHg >= 0){
        printf("%.2f hectopascales equivalen a %.2f mmHg\n", hPa, mmHg);
    } else {
        printf("Por favor, ingrese valores validos.");
    }
    return 0;
}
```

Salida del programa

```
C:\Users\ASUS TUF A16\Desktop\TDLP\David-Ruiz_teoria-de-la-programaci-n\Unidad 1\20 ejercicios>e250
Ingrese la presion en hPa: 230
230.00 hectopascales equivalen a 172.51 mmHg

C:\Users\ASUS TUF A16\Desktop\TDLP\David-Ruiz_teoria-de-la-programaci-n\Unidad 1\20 ejercicios>e250
Ingrese la presion en hPa: -124
Por favor, ingrese valores validos.
```

Conclusiones del algoritmo

El algoritmo evalúa correctamente los valores positivos y da la transformación de hectopascales a mmHg

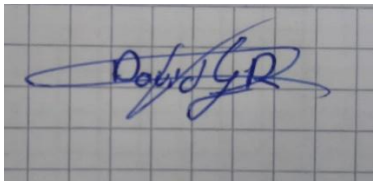
UNIVERSIDAD NACIONAL DE LOJA

Enlace del repositorio

https://github.com/David8-arch/David-Ruiz_teoria-de-la-programaci-n.git

- **Declaración de autoría**

Declaro que el presente trabajo es de mi **autoría,** que **no se utilizó inteligencia artificial** (ni otra herramienta generadora de contenido) ni copia de terceros para su elaboración, y que conozco y acepto las sanciones establecidas en el reglamento institucional en caso de incumplimiento.



Firma del estudiante **Docente**